

DEBTREX

Internal System

Developer Documentation

A complete guide to editing, extending, and operating the DEBTREX internal operations platform. Covers architecture, every feature, every API, every database table, the permission system, and copy-paste recipes for common changes.

Built with Next.js 14 · TypeScript · Supabase · Tailwind CSS

Table of Contents

#	Document	What it covers
1	Documentation Index README.md	The index. Start here if you're new.
2	The Editing Guide EDITING-GUIDE.md	The master guide. Mental model + every common edit.
3	Feature Walkthroughs feature-walkthroughs.md	Every feature: files, tables, permissions, common edits.
4	Recipes — Copy-Paste Templates recipes.md	30 copy-paste templates for common tasks.
5	File Index file-index.md	Every file in the project mapped to its purpose.
6	How I Built: Task Requests HOW-I-BUILT-task-requests.md	Step-by-step record of building the task-requests feature.
7	Architecture architecture.md	High-level shape of the system.
8	Data Model data-model.md	Every database table and how they relate.
9	Auth & Permissions auth-and-permissions.md	Login flow, role system, the permission key matrix.
10	API Reference api-reference.md	Every API route at a glance.
11	UI & Styling ui-and-styling.md	Tailwind setup, brand classes, UI patterns.
12	Developer Workflow dev-workflow.md	Running locally, env vars, common tasks.

13	Performance Notes performance.md	What was slow, what was fixed, where to look next.
14	Troubleshooting troubleshooting.md	Every error → cause → fix.

PART 1

Documentation Index

Source: docs/README.md

DEBTREX Internal System — Developer Documentation

This folder is the dev-facing complement to the top-level `README.md` (which covers operator setup).

New to the codebase? Start with [EDITING-GUIDE.md](#). It's the master document — read that one and you can edit 95% of the codebase confidently.

All the docs

Read these first

#	Doc	What you'll learn
1	EDITING-GUIDE.md	The master guide. Mental model, file map, page/API anatomy, the 12 most common edits as walkthroughs, full "build a new feature from scratch" recipe, common errors. Start here.
2	feature-walkthroughs.md	Every feature explained: files it uses, tables it touches, permissions it needs, common edits.
3	recipes.md	30 copy-paste-friendly templates for common tasks (add a column, gate a route, send an email, etc.).
4	file-index.md	Every file in the project mapped to its purpose.
5	HOW-I-BUILT-task-requests.md	Blow-by-blow record of building a real feature. Use as a reference template for any approval workflow.

Reference (deeper dives)

#	Doc	What it covers
5	architecture.md	High-level shape of the system — Next.js App Router, server vs. client, Supabase, external services
6	data-model.md	Every database table, what it stores, and how the tables relate

#	Doc	What it covers
7	auth-and-permissions.md	Login flow, session cookies, role system, the permission key matrix
8	api-reference.md	Every API route, what method, what it does, who can call it
9	ui-and-styling.md	Tailwind setup, brand colors, button/card/badge classes, the page shell pattern
10	dev-workflow.md	Running locally, env vars, common dev tasks, gotchas
11	performance.md	The perf model — what was slow, what was fixed, where to look next
12	troubleshooting.md	Every error we've seen → cause → fix

Quick "I want to do X" router

Use this to jump straight to the right doc.

I want to...	Go here
Understand the whole system	EDITING-GUIDE.md § 1–4
Add a new page	EDITING-GUIDE § 5 + recipes.md → R14
Add a new API endpoint	EDITING-GUIDE § 6 + recipes.md → R7–R10
Add a database column	EDITING-GUIDE § 8 + recipes.md → R1
Add a new role	/roles admin UI — no code needed
Add a new permission key	EDITING-GUIDE § 7.4
Add a sidebar link	recipes.md → R27
Send a templated email	recipes.md → R24
Build a brand-new feature from scratch	EDITING-GUIDE § 12
Find which file does X	file-index.md
Find which permission gates X	feature-walkthroughs.md (look up the feature)
Fix an error message I'm seeing	troubleshooting.md
Change a color or font	ui-and-styling.md

The codebase at a glance

debtrex-system/	
■ ■ ■ ■ app/	Next.js App Router – pages + API routes
■ ■ ■ ■ api/	Backend route handlers (route.ts files)
■ ■ ■ ■ (every other dir)/	A page route – page.tsx is the entry
■ ■ ■ ■ layout.tsx	Root layout (just html/body shell)
■ ■ ■ ■ components/	Shared client components (Sidebar, TopBar, dialers)
■ ■ ■ ■ lib/	Server-side utilities – auth, permissions, supabase client, integrations
■ ■ ■ ■ database/	schema.sql + numbered migration-NNN.sql files
■ ■ ■ ■ middleware.ts	Auth gate – runs on every request before page/API handlers
■ ■ ■ ■ docs/	You are here.

The 4 places you'll edit most often

- **A new page** → `app/<route>/page.tsx`. Server component by default. See [EDITING-GUIDE § 5](#).
- **A new API endpoint** → `app/api/<route>/route.ts`. Export GET/POST/PATCH/DELETE. See [EDITING-GUIDE § 6](#).
- **A new database column or table** → add a new `database/migration-NNN.sql` file. **Don't edit** `schema.sql` for live changes; it's the initial-install reference.
- **A new permission key** → add to `lib/permissions.ts` AND seed it in a new migration. See [EDITING-GUIDE § 7.4](#).

Tech stack at a glance

Concern	Choice
Framework	Next.js 14 (App Router)
Language	TypeScript
Styling	Tailwind CSS
Database	PostgreSQL via Supabase
Auth	JWT cookies (custom — not Supabase Auth)
Email	Resend
File storage	Google Drive (per-user OAuth)
Voice	Twilio (browser SDK + webhooks)
Calendar UI	FullCalendar
Icons	lucide-react

Conventions, cheat-sheet style

Thing	Convention
Files	kebab-case for routes, PascalCase for components, camelCase for utilities
Routes (URLs)	/snake_case for paths, [id] for dynamic segments
DB tables	snake_case, plural (tasks, pipeline_contacts)
DB columns	snake_case
Money	numeric(12,2) in SQL, plain number in TS — never float
Timestamps	timestampz with default now()
Primary keys	uuid
Authorization	Always app-level (hasPermission). RLS is OFF — never assume the DB will block bad calls.
Server-only files	import 'server-only' at the top
Client components	"use client" as the first line

If you only remember one rule: **every API route must call `getCurrentUser()` first and check permissions before doing any DB work.**

PART 2

The Editing Guide

Source: docs/EDITING-GUIDE.md

The Editing Guide — How to Add and Edit Functions Yourself

This is the master document. If you read only one doc, read this one. It teaches you the mental model, the file map, and the exact recipes for the changes you'll make 95% of the time.

Table of contents

1. [The mental model in 60 seconds](#)
 2. [The 3 layers you'll touch](#)
 3. [Project file map — every folder explained](#)
 4. [The standard request lifecycle \(what happens when a user clicks\)](#)
 5. [How a page is built \(anatomy\)](#)
 6. [How an API route is built \(anatomy\)](#)
 7. [The permission system, end to end](#)
 8. [Database — how to read and change it](#)
 9. [Server components vs client components — the cheat sheet](#)
 10. [The Supabase query language you actually need](#)
 11. [The 12 most common edits \(recipes\)](#)
 12. [Adding a brand-new feature from scratch](#)
 13. [Common errors and what they mean](#)
 14. [Glossary](#)
-

1. The mental model in 60 seconds

DEBTREX is **one Next.js app** that serves both the website (pages) and the backend (API). They live in the **same folder** — `app/`. The split is just by filename:

- `app/.../page.tsx` → a webpage someone visits in the browser.
- `app/api/.../route.ts` → an HTTP endpoint that returns JSON.

Both run on the server first. Both use the same auth system, the same database, the same helpers. Code in `lib/` is shared utilities. Code in `components/` is shared React UI.

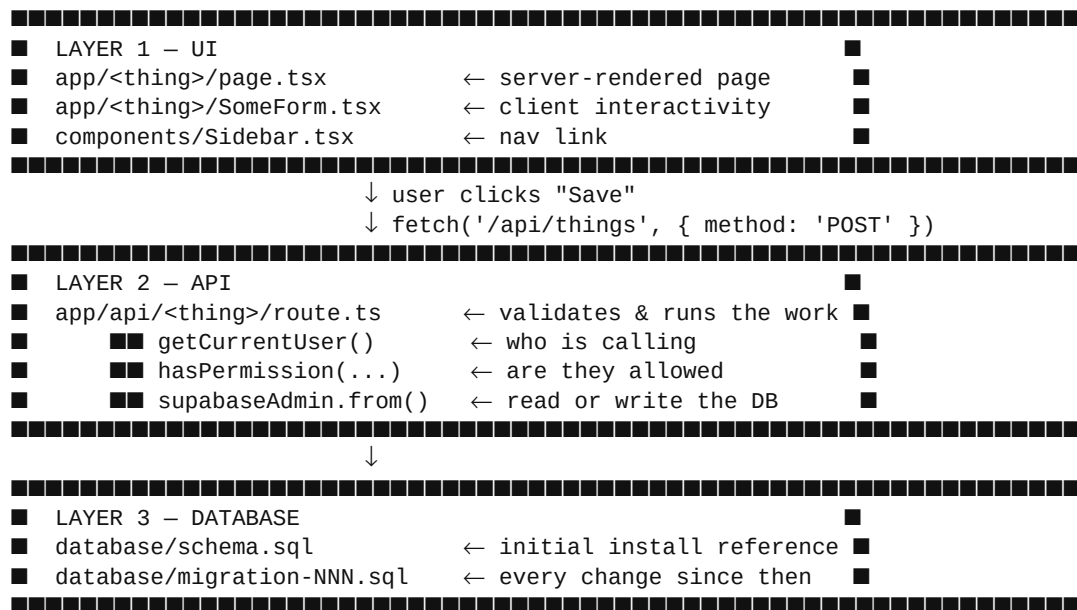
The database is **PostgreSQL on Supabase**. We talk to it through `supabaseAdmin` — a server-side client that bypasses any row-level security. **Security is enforced in our TypeScript code**, not at the DB level. That means every API route **MUST** check who's calling and whether they're allowed.

Authentication is a **cookie called** `debtrex_session`, which contains a signed JWT (basically a tamper-proof ID card). Every request shows this cookie at the door (`middleware.ts`), and every page/route re-checks it via `getCurrentUser()`.

That's it. Everything else is detail.

2. The 3 layers you'll touch

When you add or edit a "function" (in the website sense — a feature), you'll usually touch some combination of three layers:



Rule of thumb for which layers to touch:

What you're doing	Layers
Rename a button label	1 only
Add a new field to a form (saves to existing column)	1
Add a new field that needs storage	1 + 2 + 3
Allow a new role to access an existing page	nothing — toggle in /permissions UI
Add a new permission key	1 (lib/permissions.ts + admin UI) + 2 (use it) + 3 (seed it)
Add a whole new module	all 3

3. Project file map — every folder explained

```

debtrex-system/
├── app/                                ← THE WHOLE WEBSITE. Pages + API.
│   ├── api/                            ← Backend HTTP endpoints
│   │   ├── auth/                       ← login, logout, first-CEO setup
│   │   ├── budgets/                   ← monthly budget allocations
│   │   ├── calculations/              ← saved calculator scenarios
│   │   ├── events/                   ← calendar events
│   │   ├── expenses/                 ← expense submit / approve
│   │   ├── files/                    ← Google Drive upload / delete
│   │   ├── google/                   ← OAuth start + callback
│   │   ├── income/                   ← record income
│   │   ├── manager-assignments/       ← manager↔agent links
│   │   ├── me/                       ← stuff about the calling user (permissions, status)
│   │   ├── permissions/              ← bulk-update role permission matrix
│   │   ├── pipeline/                 ← contacts, notes, collaborators, call logs
│   │   ├── roles/                    ← create / edit / delete custom roles
│   │   ├── tasks/                    ← task CRUD + updates + notes
│   │   ├── twilio/                   ← phone calls – see the Twilio guide
│   │   └── users/                    ← invite, edit, deactivate, change role
│   ├── login/                         ← /login page (the only un-authed page)
│   ├── page.tsx                       ← / (the dashboard)
│   ├── tasks/                         ← /tasks list, /tasks/[id], /tasks/new
│   ├── calendar/                     ← /calendar
│   ├── files/                        ← /files
│   ├── budget/                       ← /budget + /budget/allocations
│   ├── calculators/                  ← /calculators (DTI + budget calculators)
│   ├── pipeline/                     ← /pipeline/sales/* and /pipeline/management/*
│   ├── performance/                 ← /performance + /performance/assignments
│   ├── permissions/                  ← /permissions admin (toggle the matrix)
│   ├── roles/                       ← /roles admin (create custom roles)
│   ├── team/                        ← /team – invite / deactivate members
│   ├── twilio-numbers/               ← /twilio-numbers admin
│   ├── settings/                     ← /settings (current user's profile)
│   ├── notifications/                ← /notifications inbox
│   ├── layout.tsx                    ← root HTML shell (just <html><body>)
│   └── globals.css                   ← Tailwind base + brand custom CSS
├── components/                       ← Shared client components used across pages
│   ├── Sidebar.tsx                  ← the left navigation; reads permissions
│   ├── TopBar.tsx                   ← header bar with status dropdown
│   ├── BrowserDialer.tsx            ← in-browser Twilio outbound call widget
│   └── InboundCallListener.tsx       ← invisible mount that receives inbound calls
├── lib/                              ← Shared server-side utilities (SECRETS HERE)
│   ├── auth.ts                      ← getCurrentUser, JWT sign/verify, cookie helpers
│   ├── roles.ts                     ← canManageUsers(), canEditBudget(), etc – safe on client too
│   ├── permissions.ts                ← hasPermission(), getPermissions() + permission key list
│   ├── supabase.ts                  ← the two DB clients (anon + admin)
│   ├── email.ts                     ← Resend wrapper + every email template
│   ├── google-drive.ts              ← Drive OAuth + upload/download
│   ├── twilio.ts                    ← server-side Twilio REST helpers
│   ├── twilio-voice.ts              ← browser Voice SDK token minting
│   └── performance.ts               ← KPI calculations for the performance page

```

```

■
■■■ database/
■   ■■■ schema.sql      ← INITIAL install (reference snapshot)
■   ■■■ migration-NNN.sql ← every change applied after launch, in order
■
■■■ middleware.ts        ← the door – runs before every request, gates by cookie
■■■ next.config.js       ← Next.js build settings
■■■ tailwind.config.js   ← Tailwind palette (brand-red, brand-ink, etc.)
■■■ package.json         ← dependencies + npm scripts
■■■ docs/                ← you are here

```

What about `node_modules/` and `.next/`?

You never edit those. `node_modules/` is your installed dependencies (managed by `npm install`). `.next/` is Next.js's build cache (delete it if dev mode acts weird; it'll rebuild).

4. The standard request lifecycle

Whenever a user clicks anything, this is what happens. Memorize the shape:

A) When they navigate to a page (e.g. `/tasks`)

Browser → GET `/tasks`

```

■
■■■ middleware.ts          ← reads `debtrex_session` cookie
■   ■■ no cookie? → redirect to /login
■   ■■ bad cookie? → redirect to /login
■   ■■ good cookie → continue
■
■■■ app/tasks/page.tsx     ← server React component, async
■   ■■ getCurrentUser()    ← re-verifies JWT and re-fetches profile
■   ■■ (optional) permission check → redirect('/') if denied
■   ■■ supabaseAdmin.from('tasks').select(...) ← runs queries
■   ■■ returns JSX with the data
■
■■■ HTML streams back to the browser
■■■ Client components ("use client") hydrate for interactivity

```

B) When their form makes an API call (e.g. POST `/api/tasks`)

Browser → fetch(`/api/tasks`, { method: 'POST', body: JSON })

```

■
■■■ middleware.ts          ← same cookie check; returns 401 JSON if denied
■
■■■ app/api/tasks/route.ts ← exported `POST` function runs
■   ■■ getCurrentUser()    ← returns 401 if null
■   ■■ hasPermission(...)  ← returns 403 if false (sometimes)
■   ■■ validate body       ← reject 400 if input is wrong
■   ■■ supabaseAdmin.insert() ← do the write
■   ■■ side effects        ← send email, write audit_log, create notification
■   ■■ return NextResponse.json(data)
■
■■■ Browser gets back JSON; usually re-renders or router.refresh()

```

This pattern repeats **everywhere**. Once you've seen it twice, you can read any route in the project.

5. How a page is built (anatomy)

Every page in `app/<route>/page.tsx` looks roughly the same. Here's the canonical pattern, annotated.

```
// app/widgets/page.tsx
import { redirect } from 'next/navigation';
import { getCurrentUser } from '@lib/auth';
import { hasPermission } from '@lib/permissions';
import { supabaseAdmin } from '@lib/supabase';
import Sidebar from '@components/Sidebar';
import TopBar from '@components/TopBar';
import Link from 'next/link';
import { Plus } from 'lucide-react';
import WidgetForm from './WidgetForm'; // client component, see step 5b

// 1. The default export is the page. It's an ASYNC function – that's what
//    makes this a Server Component. Async + no "use client" = runs on server.
export default async function WidgetsPage() {

  // 2. Auth: who is asking? Redirect if not signed in.
  const user = await getCurrentUser();
  if (!user) redirect('/login');

  // 3. Permission: can they even see this page? Redirect home if not.
  //    (We "use the granular system OR fall back to a hard-coded helper".)
  const allowed = await hasPermission(user.role, 'section.widgets');
  if (!allowed) redirect('/');

  // 4. Fetch your data. Use Promise.all to parallelize independent queries.
  const [widgetsRes, peopleRes] = await Promise.all([
    supabaseAdmin
      .from('widgets')
      .select('id, name, status, created_at')
      .order('created_at', { ascending: false })
      .limit(200), // always bound the result set
    supabaseAdmin
      .from('profiles')
      .select('id, full_name')
      .eq('is_active', true),
  ]);

  const widgets = widgetsRes.data || [];
  const people = peopleRes.data || [];

  // 5. Return the JSX. Sidebar + TopBar are reused on every page.
  return (
    <div className="flex min-h-screen bg-gray-50">
      <Sidebar user={user} />
      <main className="flex-1 min-w-0">
        <TopBar user={user} title="Widgets" />
        <div className="p-4 sm:p-6 max-w-7xl mx-auto">

          { /* Header row */ }
```

```

<div className="flex items-center justify-between mb-6">
  <h1 className="font-condensed text-2xl font-black uppercase">All Widgets</h1>
  <Link href="/widgets/new" className="btn-primary">
    <Plus size={14} /> New Widget
  </Link>
</div>

{/* List */}
{widgets.length === 0 ? (
  <p className="text-gray-500">No widgets yet.</p>
) : (
  <ul>
    {widgets.map(w => <li key={w.id}>{w.name}</li>)}
  </ul>
)}

{/* Interactive form is a client component – see below */}
<WidgetForm people={people} />
</div>
</main>
</div>
);
}

```

5b. The client component (WidgetForm.tsx)

Whenever you need `useState`, `useEffect`, event handlers, or anything reactive, that piece must be a Client Component. Mark it with `"use client"` at the very top.

```

// app/widgets/WidgetForm.tsx
"use client";
import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function WidgetForm({ people }: { people: { id: string; full_name: string }[] }) {
  const router = useRouter();
  const [name, setName] = useState('');
  const [saving, setSaving] = useState(false);

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setSaving(true);
    const res = await fetch('/api/widgets', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name }),
    });
    setSaving(false);

    if (!res.ok) {
      const err = await res.json();
      alert(err.error || 'Failed to save');
      return;
    }
    setName('');
    router.refresh(); // re-runs the server component → shows the new widget
  }

  return (

```

```

<form onSubmit={handleSubmit} className="card p-4 mt-6">
  <input
    value={name}
    onChange={e => setName(e.target.value)}
    placeholder="Widget name"
    className="input"
  />
  <button type="submit" disabled={saving} className="btn-primary mt-3">
    {saving ? 'Saving...' : 'Save'}
  </button>
</form>
);
}

```

Why split into two files? Server components can be `async` and talk to the DB. Client components can have state and handlers. You can't do both in one component. So the pattern is: server page fetches data, passes it as props to a client component.

6. How an API route is built (anatomy)

Every API endpoint lives at `app/api/<path>/route.ts`. The file's exports define which HTTP verbs work:

```

export async function GET(req)    { ... }
export async function POST(req)   { ... }
export async function PATCH(req)  { ... }
export async function DELETE(req) { ... }

```

Here's the canonical POST route, fully annotated:

```

// app/api/widgets/route.ts
import { NextResponse } from 'next/server';
import { getCurrentUser } from '@lib/auth';
import { hasPermission } from '@lib/permissions';
import { supabaseAdmin } from '@lib/supabase';

export async function POST(request: Request) {
  // 1. AUTH – always first.
  const user = await getCurrentUser();
  if (!user) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  // 2. PERMISSION – always second.
  if (!(await hasPermission(user.role, 'widget.create'))) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  // 3. PARSE + VALIDATE input. Never trust the body shape.
  let body: any;
  try {
    body = await request.json();
  } catch {
    return NextResponse.json({ error: 'Invalid JSON' }, { status: 400 });
  }
}

```



```

const { name, color } = body;
if (!name || typeof name !== 'string') {
  return NextResponse.json({ error: 'name is required' }, { status: 400 });
}

// 4. DO THE WORK. supabaseAdmin = service-role; bypasses RLS.
const { data, error } = await supabaseAdmin
  .from('widgets')
  .insert({
    name,
    color: color || 'gray',
    created_by: user.id,          // always stamp who did it
  })
  .select()                     // return the new row
  .single();

if (error) {
  return NextResponse.json({ error: error.message }, { status: 500 });
}

// 5. SIDE EFFECTS – audit log, notifications, emails. Optional.
await supabaseAdmin.from('audit_log').insert({
  user_id: user.id,
  action: 'widget_created',
  resource_type: 'widget',
  resource_id: data.id,
  details: { name },
});

// 6. RETURN. Always JSON.
return NextResponse.json(data);
}

```

Dynamic routes (the `[id]` thing)

If your URL has an ID in it (`/api/widgets/abc-123`), the file goes at `app/api/widgets/[id]/route.ts`. You read the ID from `params`:

```

export async function PATCH(
  request: Request,
  { params }: { params: Promise<{ id: string }> } // Next.js 15 wraps params in a Promise
) {
  const { id } = await params;
  // ... rest of the handler ...
}

```

Status codes — keep it simple

Code	Meaning	When to use
200	OK	Success — return JSON
400	Bad Request	The body is wrong / missing fields
401	Unauthorized	No cookie / invalid cookie
403	Forbidden	Logged in but not allowed

Code	Meaning	When to use
404	Not Found	The resource doesn't exist
500	Server Error	The DB failed, or anything we didn't anticipate

7. The permission system, end to end

There are **three different "permission" mechanisms** in the codebase. They sound similar but solve different problems. **Memorize this section** — it's where most confusion lives.

7.1 Built-in role helpers (`lib/roles.ts`)

Hard-coded TypeScript functions. They take a role string and return `true/false`. Examples:

```
canManageUsers('ceo')    // true
canEditBudget('manager') // false
isLeadership('co-owner') // true
```

These are **fast** (no DB hit) and **always work for built-in roles**. They do NOT know about custom roles — a custom role will always return `false`.

Use them as fallbacks or for simple binary checks.

7.2 Granular permission keys (`lib/permissions.ts` + `role_permissions` table)

Every (role, permission_key) pair has an `enabled` boolean in the `role_permissions` table. The CEO can toggle any of them on the `/permissions` page in the UI.

There are ~50 keys — see `PERMISSION_KEYS` in `lib/permissions.ts`. Examples:

```
section.tasks    → can see Tasks nav item + page
task.create      → can create tasks
task.delete      → can delete tasks
pipeline.view_all → can see contacts not assigned to them
expense.approve  → can approve / reject expenses
```

You check them like this:

```
import { hasPermission } from '@lib/permissions';

if (await hasPermission(user.role, 'task.create')) {
  // they can do it
}
```

Reading is cached for 5 minutes in the Node process memory. So calling this 1000 times only hits the DB once (per process).

After you change permissions in the admin UI, the API calls `invalidatePermissionCache()` so the next request reads fresh data.

7.3 Per-user / per-resource checks (inline)

Sometimes the question isn't "can this role do X" but "can THIS user touch THIS row." Examples:

- You can edit your own task even if you don't have `task.edit_any`.
- You can delete your own note even if you're not the author.
- You can see a pipeline contact if you're assigned OR a collaborator.

These checks live inside the specific route handler:

```
const task = await supabaseAdmin.from('tasks').select('created_by, assigned_to').eq('id', id).single();
const isOwner = task.data?.created_by === user.id || task.data?.assigned_to === user.id;
if (!isOwner && !(await hasPermission(user.role, 'task.edit_any'))) {
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
}
```

7.4 Adding a new permission — full walkthrough

Let's add `widget.delete`.

Step 1 — Add it to the key list. Open `lib/permissions.ts`:

```
export const PERMISSION_KEYS = [
  // ... existing keys ...
  'widget.delete', // ← add here
] as const;
```

Step 2 — Add a label. Same file, `PERMISSION_LABELS`:

```
export const PERMISSION_LABELS: Record<PermissionKey, string> = {
  // ... existing ...
  'widget.delete': 'Delete widgets',
};
```

Step 3 — Put it in a group so the admin UI shows it under the right section:

```
export const PERMISSION_GROUPS = [
  // ... existing ...
  {
    label: 'Widgets',
    keys: ['widget.create', 'widget.delete'],
  },
];
```

Step 4 — Seed it in a migration. Create `database/migration-008.sql` (next number after the last one):

```
-- migration-008.sql
-- Seed widget.delete for all built-in roles.
do $$
declare
  r text;
  default_enabled boolean;
begin
  for r in select unnest(array['ceo', 'owner', 'co-owner', 'manager', 'employee', 'accountant', 'viewer']) loop
    -- only enable for leadership
    default_enabled := r in ('ceo', 'owner', 'co-owner');
    insert into role_permissions(role, permission_key, enabled)
    values (r, 'widget.delete', default_enabled)
    on conflict (role, permission_key) do nothing;
  end loop;
```

```
end$$;
```

Step 5 — Run the migration. Open Supabase Dashboard → SQL Editor → paste the migration → Run.

Step 6 — Use it in your DELETE route:

```
if (!(await hasPermission(user.role, 'widget.delete'))) {
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
}
```

Step 7 (optional) — Show/hide a UI element based on it. The Sidebar already does this — see how it calls `can('section.widgets', fallback)`. For other places, you can fetch `/api/me/permissions` once and check the map.

That's it. The key is now toggleable in `/permissions` for any role, including custom ones.

8. Database — how to read and change it

8.1 The two SQL files you care about

File	Purpose	When to edit
<code>database/schema.sql</code>	The initial install snapshot . If someone clones the project and runs this in a fresh Supabase project, they get a working schema as of the day it was written.	Almost never. Don't add new columns here mid-flight — that's what migrations are for.
<code>database/migration-NNN.sql</code>	An incremental change . One file per logical change set, numbered in order.	Always — every DB change you make goes in a new migration file.

8.2 Adding a column

Say you want to add a `priority` column to the `widgets` table.

1. Look at `ls database/` to find the highest migration number. The next one is `migration-008.sql`.
2. Create the file:

```
-- migration-008.sql
-- Adds a priority column to widgets.
alter table widgets
  add column if not exists priority text default 'normal'
  check (priority in ('low', 'normal', 'high'));
```

3. Open Supabase → SQL Editor → paste → Run.
4. Now use it in your code:

```
.select('id, name, priority')
.insert({ name, priority: 'high' })
```

8.3 Adding a new table

```
-- migration-009.sql
create table if not exists widgets (
  id uuid default uuid_generate_v4() primary key,
  name text not null,
  color text default 'gray',
  created_by uuid references profiles(id) not null,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- One index per column the app filters on
create index if not exists widgets_created_by_idx on widgets(created_by);

-- If you want auto-updating updated_at:
create trigger update_widgets_updated_at before update on widgets
  for each row execute function update_updated_at_column();
```

Always include `if not exists` so re-running the migration is safe.

8.4 Database conventions

Topic	Rule
Primary keys	<code>uuid default uuid_generate_v4() primary key</code>
Timestamps	<code>timestamptz default now()</code>
Money	<code>numeric(12,2)</code> — never <code>float</code> , never <code>real</code>
Enums	text with a check <code>(column in (...))</code> constraint. To add a value, alter the constraint in a migration.
Foreign keys	<code>references some_table(id)</code> . Use on delete cascade when child rows should die with the parent (e.g. <code>task_updates</code> when a task is deleted). Use nothing for <code>created_by</code> (you don't want deleting a user to nuke their work).
Soft delete	<code>is_active = false</code> on profiles. Everything else hard-deletes.
Indexes	One index per column you <code>.eq()</code> filter on. Composite index when you filter on two columns together.

8.5 How to actually run a query (in your code)

```
// SELECT
const { data, error } = await supabaseAdmin
  .from('widgets')
  .select('id, name, color')          // ← only the columns you need
  .eq('created_by', user.id)         // WHERE created_by = user.id
  .order('created_at', { ascending: false })
  .limit(50);

// INSERT (returns the new row when you add .select().single())
const { data, error } = await supabaseAdmin
  .from('widgets')
  .insert({ name: 'Foo', color: 'blue', created_by: user.id })
  .select()
  .single();

// UPDATE
const { error } = await supabaseAdmin
  .from('widgets')
  .update({ color: 'red' })
  .eq('id', widgetId);

// DELETE
const { error } = await supabaseAdmin
  .from('widgets')
  .delete()
  .eq('id', widgetId);
```

Always check `error`. If there is one, return 500.

8.6 Joining tables (the `select()` trick)

To join `widgets` to `profiles` on `created_by` and get the creator's name in the same query:

```
.select(`
  id, name,
  creator:profiles!widgets_created_by_fkey(id, full_name)
`)
```

The string before the colon (`creator`) becomes the property name on each row. The thing after the `!` is the FK constraint name. (Look at your migration to see what Supabase named it — typically

`<table>_<column>_fkey`.)

9. Server components vs client components

This trips people up. Memorize this table.

Need	Type	How
Just render data from props or DB	Server	Default. Just export default <code>async function</code> . No directive.

Need	Type	How
useState, useEffect, event handlers, browser APIs	Client	First line of file: <code>"use client";</code>
Fetch data	Server	<code>await supabaseAdmin.from(...)</code> . If a Client needs data, call an API route via <code>fetch()</code> .
Show loading spinners	Client	Server components don't have "loading" — Next.js handles it via <code>loading.tsx</code> files.
Use localStorage / sessionStorage	Client	Wrap reads in <code>typeof window !== 'undefined'</code> checks to avoid SSR errors.
Import <code>lib/auth.ts</code> , <code>lib/permissions.ts</code> , <code>lib/email.ts</code>	Server ONLY	These files have <code>import 'server-only'</code> and will fail in a client component.
Import <code>lib/roles.ts</code>	Either	This file has no <code>server-only</code> — safe everywhere.
Use <code>getCurrentUser()</code>	Server	Reads cookies via <code>next/headers</code> — server only.

How to pass data from server → client

The whole point of server components is they fetch data and pass it down as JSX props:

```
// page.tsx (server)
const widgets = await supabaseAdmin.from('widgets').select('*');
return <WidgetList widgets={widgets.data ?? []} />;

// WidgetList.tsx (client)
"use client";
export default function WidgetList({ widgets }: { widgets: Widget[] }) {
  // can now useState etc.
}
```

How a Client component fetches data

It can't query Supabase directly (no service key in the browser). It calls an API route:

```
"use client";
import { useEffect, useState } from 'react';

export default function MyWidget() {
  const [data, setData] = useState<any>(null);
  useEffect(() => {
    fetch('/api/widgets').then(r => r.json()).then(setData);
  }, []);
  return <div>{data?.length}</div>;
}
```

10. The Supabase query language you actually need

Almost everything we do with the DB is one of these patterns. Bookmark this section.

Selecting

```
.select('id, name, status')           // pick columns
.select('*')                         // everything (avoid in lists – heavy)
.eq('user_id', user.id)              // =
.neq('status', 'completed')          // !=
.in('status', ['open', 'pending'])   // status IN (...)
.gt('amount', 100)                   // >
.gte('created_at', '2026-01-01')      // >=
.lt('end_time', new Date().toISOString()) // <
.like('email', '%@gmail.com')         // SQL LIKE
.ilike('name', '%john%')              // case-insensitive LIKE
.or('status.eq.open,status.eq.pending') // OR (annoying syntax – use sparingly)
.order('created_at', { ascending: false }) // ORDER BY
.limit(50)                           // LIMIT
.range(0, 49)                        // pagination (inclusive both ends)
.single()                           // expect exactly 1 row; errors if 0 or 2+
.maybeSingle()                       // 0 or 1 row; null if 0
```

Inserting

```
// Single row
const { data, error } = await supabaseAdmin
  .from('widgets')
  .insert({ name: 'Foo', created_by: user.id })
  .select() // optional – adds the new row to `data`
  .single(); // unwrap from array

// Multiple rows
await supabaseAdmin
  .from('event_attendees')
  .insert([
    { event_id: id, user_id: u1, status: 'invited' },
    { event_id: id, user_id: u2, status: 'invited' },
  ]);
```

Updating

```
await supabaseAdmin
  .from('widgets')
  .update({ color: 'red', updated_at: new Date().toISOString() })
  .eq('id', widgetId);
```

Deleting

```
await supabaseAdmin
  .from('widgets')
  .delete()
  .eq('id', widgetId);
```


Upsert (insert OR update if a unique key conflicts)

```
await supabaseAdmin
  .from('budgets')
  .upsert(
    { month: '2026-05-01', category: 'rent', allocated_amount: 5000 },
    { onConflict: 'month,category' } // matches the UNIQUE constraint
  );
```

Counting

```
const { count } = await supabaseAdmin
  .from('tasks')
  .select('*', { count: 'exact', head: true }) // head:true = don't return rows
  .eq('assigned_to', user.id);
```

11. The 12 most common edits (recipes)

These are copy-paste-friendly. Each is a self-contained mini-walkthrough.

Recipe 1 — Change a page title or button label

Open the relevant `app/<route>/page.tsx` (or component) and just edit the string. No build step.

```
// before
<TopBar user={user} title="Tasks" />
// after
<TopBar user={user} title="My Work" />
```

Recipe 2 — Add a new field to an existing form

Example: add a `notes` field to "Create Task".

1. **Add the column** (if it doesn't exist) — migration:

```
alter table tasks add column if not exists notes text;
```

2. **Add the input** in `app/tasks/new/NewTaskForm.tsx` (a client component):

```
const [notes, setNotes] = useState('');
// ...
<textarea value={notes} onChange={e => setNotes(e.target.value)} className="input" />
```

3. **Send it** in the `fetch` body:

```
body: JSON.stringify({ title, description, notes, /* ... */ })
```

4. **Accept it** in `app/api/tasks/route.ts`:

```
const { title, description, notes, /* ... */ } = body;
```

5. **Store it** in the insert:

```
.insert({ title, description, notes, /* ... */ })
```

6. **Show it** wherever you read tasks back (e.g. `app/tasks/[id]/page.tsx`) — add `notes` to the

```
.select('...') and render it.
```

Recipe 3 — Add a permission gate to an existing page

You decided `/calculators` should require a new `section.calculators_advanced` key.

1. Add the key + label + group in `lib/permissions.ts` (see [7.4](#)).
2. Seed it in a migration:

```
insert into role_permissions(role, permission_key, enabled)
select unnest(array['ceo', 'owner', 'co-owner']), 'section.calculators_advanced', true
on conflict do nothing;
```

3. At the top of `app/calculators/page.tsx`:

```
if (!(await hasPermission(user.role, 'section.calculators_advanced'))) redirect('/');
```

Recipe 4 — Add a new sidebar nav item

Open `components/Sidebar.tsx`, find `const groups: NavGroup[]`, drop your item into the right group:

```
{
  label: 'Tools',
  items: [
    { href: '/calculators', label: 'Calculators', icon: Calculator, show: can('section.calculators', user),
    { href: '/reports', label: 'Reports', icon: BarChart3, show: can('section.reports', isLeaders
  ],
},
```

The `show` field controls visibility. The pattern is `can('permission.key', fallbackBool)`:

- If permissions are loaded → use the granular system.
- If permissions haven't loaded yet → use the fallback so the user doesn't see nav flicker.

Also import your new icon at the top: `import { BarChart3 } from 'lucide-react';`

Recipe 5 — Send an email when something happens

Open `lib/email.ts`. Add a new function patterned after `sendTaskAssignedEmail`:

```
export async function sendWidgetCreatedEmail(to: string, name: string, widget: any) {
  const html = emailWrapper(
    `

## Hi ${name.split(' ')[0]},</h2> <p>A new widget was created: <strong>${widget.name}</strong>.</p>`, 'View Widget', `${APP_URL}/widgets/${widget.id}`, ); return sendEmail(to, `Widget created: ${widget.name}`, html); }


```

Then call it from your route:

```
import { sendWidgetCreatedEmail } from '@lib/email';
// after the insert:
const { data: ceoProfile } = await supabaseAdmin
  .from('profiles')
  .select('email, full_name')
  .eq('role', 'ceo')
  .single();
if (ceoProfile) {
  await sendWidgetCreatedEmail(ceoProfile.email, ceoProfile.full_name, data);
}
```

Note: if `RESEND_API_KEY` isn't set in `.env.local`, emails silently log to the console. That's by design so dev doesn't fail without keys.

Recipe 6 — Create an in-app notification

In-app notifications are rows in the `notifications` table. The bell icon in TopBar reads from it.

```
await supabaseAdmin.from('notifications').insert({
  user_id: assignee.id,           // who sees it
  type: 'widget_assigned',       // free-form string, used for filtering
  title: 'New widget assigned',
  message: `${user.full_name} assigned you "${data.name}"`,
  link: `/widgets/${data.id}`,   // where the bell click navigates
});
```

Recipe 7 — Add an audit log entry

Always do this for sensitive actions (creates, deletes, role changes, money). The audit log is append-only.

```
await supabaseAdmin.from('audit_log').insert({
  user_id: user.id,
  action: 'widget_deleted',       // verb_noun, your call
  resource_type: 'widget',
  resource_id: widgetId,
  details: { name: data.name },   // anything else as jsonb
});
```

Recipe 8 — Add a new role

You don't need code for this — use the `/roles` admin UI.

1. Sign in as CEO.
2. Go to `/roles`.
3. Click "New Role".
4. Pick a key (e.g. `intern`), a friendly label, and a color.
5. Go to `/permissions` and toggle on whatever they need.

The key is stored on `profiles.role` for users who get this role. Built-in helpers like `isLeadership()` won't recognize it (since they're hard-coded), so anything gated by those will treat it as a regular user. The granular permission keys WILL work for it.

Recipe 9 — Restrict an action to "owner or admin"

The pattern for "you can edit your own thing OR you have a global permission":

```
const { data: row } = await supabaseAdmin
  .from('widgets')
  .select('created_by')
  .eq('id', id)
  .single();

const isOwner = row?.created_by === user.id;
const isAdmin = await hasPermission(user.role, 'widget.edit_any');

if (!isOwner && !isAdmin) {
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
}
```

```
}
```

Recipe 10 — Paginate a long list

Server side, use `.range(start, end)` (both inclusive):

```
const page = parseInt(searchParams.page ?? '0', 10);
const PAGE_SIZE = 25;
const start = page * PAGE_SIZE;
const end = start + PAGE_SIZE - 1;
```

```
await supabaseAdmin.from('widgets').select('*').range(start, end);
```

Browser side, just navigate to `/widgets?page=1`, etc.

Recipe 11 — Schedule a recurring job

There's no built-in scheduler. Two options:

1. **Vercel Cron** — add to `vercel.json` (you'll need to create it):

```
{
  "crons": [
    { "path": "/api/cron/daily-overdue", "schedule": "0 8 * * *" }
  ]
}
```

Then create `app/api/cron/daily-overdue/route.ts`. **Important:** it'll be public when Vercel calls it. Add a shared secret check:

```
if (request.headers.get('Authorization') !== `Bearer ${process.env.CRON_SECRET}`) {
  return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
}
```

You'll also need to whitelist `/api/cron/` in `middleware.ts`.

2. **Supabase Edge Functions + `pg_cron`** — more involved; only if you outgrow Vercel Cron's quota.

Recipe 12 — Roll back a change

If you broke something and need to revert:

```
git log --oneline -20          # find the commit BEFORE you broke it
git revert <bad-commit-hash>   # makes a new commit that undoes the bad one
git push
```

For a DB migration that turned out wrong, write a NEW migration that undoes it (e.g. `alter table widgets drop column if exists priority;`). Don't delete the broken migration file — that history is gone in prod already.

12. Adding a brand-new feature from scratch

Full walkthrough — adding a "Reports" module.

Step 1 — Decide the data shape

What do you want to store? Let's say: every report has a title, a creator, a body (markdown), and a timestamp.

Step 2 — Write the migration

database/migration-010.sql:

```
create table if not exists reports (
  id uuid default uuid_generate_v4() primary key,
  title text not null,
  body text,
  created_by uuid references profiles(id) not null,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);
create index if not exists reports_created_by_idx on reports(created_by);

create trigger update_reports_updated_at before update on reports
  for each row execute function update_updated_at_column();
```

Run it in Supabase.

Step 3 — Add the permissions

lib/permissions.ts:

```
export const PERMISSION_KEYS = [
  // ...
  'section.reports',
  'report.create',
  'report.delete',
] as const;

PERMISSION_LABELS['section.reports'] = 'See Reports section';
PERMISSION_LABELS['report.create'] = 'Create reports';
PERMISSION_LABELS['report.delete'] = 'Delete reports';

PERMISSION_GROUPS.push({
  label: 'Reports',
  keys: ['section.reports', 'report.create', 'report.delete'],
});
```

Seed in migration-011.sql:

```
do $$
declare r text;
begin
  for r in select unnest(array['ceo','owner','co-owner','manager','employee','accountant','viewer']) loop
    insert into role_permissions(role, permission_key, enabled)
      values
        (r, 'section.reports', r in ('ceo','owner','co-owner','manager')),
        (r, 'report.create', r in ('ceo','owner','co-owner','manager')),
        (r, 'report.delete', r in ('ceo','owner','co-owner'))
    on conflict (role, permission_key) do nothing;
  end loop;
end$$;
```

Run it.

Step 4 — Build the API routes

app/api/reports/route.ts:

```
import { NextResponse } from 'next/server';
import { getCurrentUser } from '@lib/auth';
import { hasPermission } from '@lib/permissions';
import { supabaseAdmin } from '@lib/supabase';

export async function POST(request: Request) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  if (!(await hasPermission(user.role, 'report.create'))) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const { title, body } = await request.json();
  if (!title) return NextResponse.json({ error: 'Title required' }, { status: 400 });

  const { data, error } = await supabaseAdmin
    .from('reports')
    .insert({ title, body, created_by: user.id })
    .select()
    .single();

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json(data);
}
```

app/api/reports/[id]/route.ts:

```
import { NextResponse } from 'next/server';
import { getCurrentUser } from '@lib/auth';
import { hasPermission } from '@lib/permissions';
import { supabaseAdmin } from '@lib/supabase';

export async function DELETE(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const { id } = await params;
  const { data: row } = await supabaseAdmin
    .from('reports').select('created_by').eq('id', id).single();

  const isOwner = row?.created_by === user.id;
  const canDelete = await hasPermission(user.role, 'report.delete');
  if (!isOwner && !canDelete) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const { error } = await supabaseAdmin.from('reports').delete().eq('id', id);
  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json({ ok: true });
}
```

Step 5 — Build the page

app/reports/page.tsx:

```
import { redirect } from 'next/navigation';
import { getCurrentUser } from '@lib/auth';
import { hasPermission } from '@lib/permissions';
import { supabaseAdmin } from '@lib/supabase';
import Sidebar from '@components/Sidebar';
import TopBar from '@components/TopBar';
import ReportForm from './ReportForm';

export default async function ReportsPage() {
  const user = await getCurrentUser();
  if (!user) redirect('/login');
  if (!(await hasPermission(user.role, 'section.reports'))) redirect('/');

  const { data: reports } = await supabaseAdmin
    .from('reports')
    .select(`id, title, body, created_at,
      creator:profiles!reports_created_by_fkey(full_name)`)
    .order('created_at', { ascending: false })
    .limit(100);

  const canCreate = await hasPermission(user.role, 'report.create');

  return (
    <div className="flex min-h-screen bg-gray-50">
      <Sidebar user={user} />
      <main className="flex-1 min-w-0">
        <TopBar user={user} title="Reports" />
        <div className="p-4 sm:p-6 max-w-5xl mx-auto">
          {canCreate && <ReportForm />}
          <div className="mt-6 space-y-4">
            {(reports ?? []).map((r: any) => (
              <article key={r.id} className="card p-4">
                <h2 className="font-bold">{r.title}</h2>
                <p className="text-xs text-gray-500">
                  By {r.creator?.full_name} on {new Date(r.created_at).toLocaleDateString()}
                </p>
                <p className="mt-2 whitespace-pre-wrap">{r.body}</p>
              </article>
            ))}
          </div>
        </div>
      </main>
    </div>
  );
}
```

app/reports/ReportForm.tsx:

```
"use client";
import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function ReportForm() {
  const router = useRouter();
  const [title, setTitle] = useState('');
```

```

const [body, setBody] = useState('');
const [saving, setSaving] = useState(false);

async function submit(e: React.FormEvent) {
  e.preventDefault();
  setSaving(true);
  const res = await fetch('/api/reports', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title, body }),
  });
  setSaving(false);
  if (!res.ok) { alert((await res.json()).error); return; }
  setTitle(''); setBody('');
  router.refresh();
}

return (
  <form onSubmit={submit} className="card p-4 space-y-3">
    <input className="input" placeholder="Report title" value={title} onChange={e => setTitle(e.target.value)} />
    <textarea className="input" rows={6} placeholder="Body" value={body} onChange={e => setBody(e.target.value)} />
    <button type="submit" disabled={saving} className="btn-primary">
      {saving ? 'Saving...' : 'Publish report'}
    </button>
  </form>
);
}

```

Step 6 — Add it to the sidebar

In `components/Sidebar.tsx`:

```

import { FileText } from 'lucide-react';
// ... inside `groups` ...
{
  label: 'Tools',
  items: [
    /* existing items */
    { href: '/reports', label: 'Reports', icon: FileText, show: can('section.reports', false) },
  ],
},

```

Step 7 — Run it

`npm run dev`

Visit `http://localhost:3000/reports`. Done.

13. Common errors and what they mean

Error you see	Meaning	Fix
Page redirects to /login immediately after I sign in	JWT_SECRET changed between sign and verify, OR cookie domain mismatch.	Make sure <code>.env.local</code> has a stable JWT_SECRET. Restart <code>npm run dev</code> . Clear the cookie in DevTools.
"Cannot find module: server-only" in client component	You imported <code>lib/auth.ts</code> or another server-only module into a "use client" file.	Move the import to a server file. Pass the result down as props.
<code>getCurrentUser()</code> returns null in an API route	The cookie is missing/expired, OR <code>is_active = false</code> on the profile.	Sign in again. If still broken, check the <code>profiles</code> table.
403 Forbidden on a page my role should access	The role doesn't have the right permission key, OR you used the wrong key.	Sign in as CEO → <code>/permissions</code> → toggle on the key for that role. Or fix the key name in your code.
"Forbidden" on a Twilio webhook	The path isn't whitelisted in <code>middleware.ts</code> .	Add the prefix to <code>TWILIO_WEBHOOK_PREFIXES</code> .
Emails never arrive	<code>RESEND_API_KEY</code> empty (logs to console silently) or domain unverified.	Check <code>.env.local</code> . Check Resend dashboard for SPF/DKIM/DMARC.
"Cannot find module 'foo'" after I added a dep	You forgot to run <code>npm install</code> .	Run it, then restart the dev server.
<code>supabaseAdmin</code> is undefined / "service role key required"	Env var missing.	Add <code>SUPABASE_SERVICE_ROLE_KEY</code> to <code>.env.local</code> .
<code>uuid_generate_v4()</code> does not exist	The <code>uuid-oss</code> extension isn't enabled in your Supabase project.	Run <code>create extension if not exists "uuid-oss";</code> in SQL editor.
TypeScript red squiggles in the editor but build passes	<code>next.config.js</code> has <code>typescript.ignoreBuildErrors: true</code> . The build is permissive on purpose.	Fix the squiggles anyway — they signal bugs that just don't block deploy.
A page loads slowly	Probably serial DB queries. See <code>docs/performance.md</code> . Use <code>Promise.all</code> .	
A page shows stale data after I edited something	Browser is rendering a cached server response. After your <code>fetch()</code> use <code>router.refresh()</code> (client) or <code>revalidatePath()</code> (server action).	

14. Glossary

- **Server Component** — A React component that runs on the server, never in the browser. Can be async. Cannot have state/handlers. Default in Next.js App Router.
- **Client Component** — A React component with `"use client"` at the top. Hydrates in the browser. Can use `useState`, etc. Cannot be async by default.
- **Middleware** — Code in `middleware.ts` that runs on every request before any page/route handler. We use it for the cookie gate.
- **API route** — A file at `app/api/<path>/route.ts` that exports HTTP-verb functions (`GET`, `POST`, etc.). Returns JSON.
- **Service role key** — A Supabase API key that bypasses Row-Level Security. Server-only. Never goes to the browser.
- **Anon key** — A public Supabase key safe to expose in the browser. We barely use it.
- **JWT** — A signed string that carries claims like `{ id, email, role }`. Tamper-proof but readable; never put secrets inside one.
- **RLS** — Row-Level Security. PostgreSQL feature for per-row access rules. **We don't use it.** Auth happens in TypeScript.
- **Migration** — A SQL file in `database/migration-NNN.sql` that incrementally changes the schema. Run in order.
- **Hydration** — The process where client components attach to server-rendered HTML and start being interactive.
- **Twiml** — Twilio's XML "instruction language" for what to do on a call (dial, record, say, etc.). Some Twilio webhooks return Twiml; we generate it in `lib/twilio*.ts`.
- **Granular permission** — A row in `role_permissions` keyed by `(role, permission_key)`. Toggleable in the admin UI.
- **Built-in helper** — A hard-coded TS function in `lib/roles.ts` like `canManageUsers(role)`. Used for fallbacks.

Where to go next

- [feature-walkthroughs.md](#) — Every feature explained: what files, what tables, what permissions.
- [api-reference.md](#) — Every API endpoint at a glance.
- [data-model.md](#) — Every database table.
- [auth-and-permissions.md](#) — Deep dive on the security model.
- [troubleshooting.md](#) — Error → cause → fix.

PART 3

Feature Walkthroughs

Source: docs/feature-walkthroughs.md

Feature Walkthroughs

This doc walks you through every feature of the website. For each one you get:

- What the user sees
- Which files run when they use it
- Which database tables are touched
- Which permissions gate it
- The most common edits you'll make

If you want the higher-level "how to build a new feature" recipe, see [EDITING-GUIDE.md § 12](#).

Index

1. [Authentication & Sessions](#)
 2. [Dashboard \(/\)](#)
 3. [Tasks \(/tasks\)](#)
 - 3b. [Task Requests \(/tasks/requests\)](#)
 4. [Calendar \(/calendar\)](#)
 5. [Pipeline — Sales & Management \(/pipeline/*\)](#)
 6. [Phone calls — outbound + inbound \(Twilio\)](#)
 7. [Budget — Income, Expenses, Allocations \(/budget\)](#)
 8. [Files & Google Drive \(/files\)](#)
 9. [Performance & Manager Assignments \(/performance\)](#)
 10. [Calculators \(/calculators\)](#)
 11. [Team \(/team\)](#)
 12. [Custom Roles \(/roles\)](#)
 13. [Permissions Admin \(/permissions\)](#)
 14. [Twilio Numbers Admin \(/twilio-numbers\)](#)
 15. [Notifications \(/notifications\)](#)
 16. [Settings \(/settings\)](#)
 17. [Status & Presence \(in TopBar\)](#)
-

1. Authentication & Sessions

What the user sees

- `/login` — email + password form.

- Successful login → redirect to `/`.
- Anything else without a cookie → bounced to `/login`.
- After 8 hours of no activity, their cookie expires and they're bounced back.

Files

File	Role
<code>app/login/page.tsx</code>	The login form (client).
<code>app/api/auth/login/route.ts</code>	Validates email/password, signs JWT, sets cookie.
<code>app/api/auth/logout/route.ts</code>	Clears the cookie.
<code>app/api/auth/setup/route.ts</code>	Creates the first CEO. Refuses if any users exist.
<code>middleware.ts</code>	Checks the cookie on every other request.
<code>lib/auth.ts</code>	<code>getCurrentUser()</code> , <code>createSessionToken()</code> , <code>verifyPassword()</code> , etc.

Tables

- `profiles` — `email`, `password_hash` (bcrypt cost 10), `is_active`, `last_login_at`.
- `audit_log` — every login is recorded.

Permissions

- None. The login flow is public. Inactive accounts are blocked by `is_active = false` on the profile.

Common edits

- **Change the session length.** `lib/auth.ts` → `expiresIn: '8h'`. Also update `maxAge: 8 * 60 * 60` in `attachSessionCookie`.
- **Add 2FA.** Significant change — you'd add a `totp_secret` column, a verification step in `/api/auth/login`, and a UI. Not done today.
- **Force users to re-login when their role changes.** Already happens automatically — `getCurrentUser()` re-fetches the profile every request, so a deactivated user is kicked on the very next nav.
- **Change the cookie name.** `lib/auth.ts` → `COOKIE_NAME`. Also update `middleware.ts` references. (Don't bother — there's no reason to.)

2. Dashboard (/)

What the user sees

- 4 KPI cards: open tasks, in-progress count, overdue count, today's events.
- A list of their open tasks (5 most due soon).
- Today's events.
- Recent activity (their last 3 task updates).

- Quick action buttons.

Files

- `app/page.tsx` — the whole dashboard. Pure server component. No client wrapper.

Tables

- `tasks` (filtered by `assigned_to = me`)
- `task_updates` (filtered by `user_id = me`)
- `events` (today's only)

Permissions

- `section.dashboard` to even see it (defaults to true for everyone).

Common edits

- **Add another KPI card.** Edit `app/page.tsx`. The `KPICard` component is right at the bottom — just call it with the new label/value. If the value needs DB data, add another `supabaseAdmin.from(...)` to the `Promise.all`.
- **Show more / fewer tasks.** Change `.limit(5)` in the tasks query.
- **Show tasks from team, not just self.** Remove the `.eq('assigned_to', user.id)` filter and adjust based on role.
- **Change KPI colors.** Look at function `KPICard` near the bottom — the `colors` object maps red/blue/gray to Tailwind classes.

3. Tasks (/tasks)

What the user sees

- A grid of tasks with status / priority badges.
- Filter tabs: "Assigned to Me", "Created by Me", "All Tasks" (only for leadership/manager).
- Detail page at `/tasks/[id]` — task fields, work log (updates), notes.
- "New Task" form at `/tasks/new`.

Files

File	Role
<code>app/tasks/page.tsx</code>	List page. Filters via query string.
<code>app/tasks/new/page.tsx</code> + <code>NewTaskForm.tsx</code>	Create form.
<code>app/tasks/[id]/page.tsx</code>	Detail view.
<code>app/tasks/[id]/TaskWorkTracker.tsx</code>	Client-side timer + status updates.
<code>app/api/tasks/route.ts</code>	POST — create a task.
<code>app/api/tasks/[id]/route.ts</code>	PATCH — update fields (status, etc).
<code>app/api/tasks/[id]/updates/route.ts</code>	POST — add a work-log entry.

File	Role
app/api/tasks/[id]/notes/route.ts	POST — add a discussion note.
lib/email.ts	sendTaskAssignedEmail, sendTaskOverdueEmail.

Tables

- `tasks` — main row.
- `task_updates` — every status change and work-log entry.
- `task_notes` — free-form discussion.
- `notifications` — bell-icon entry when assigned.
- `audit_log` — every create.

Permissions

- `section.tasks` — see the section.
- `task.create` — create new tasks.
- `task.edit_any` — edit tasks you didn't create.
- `task.delete` — delete (currently no API exposes delete).
- `task.assign_to_anyone` — assign to others, not just self.
- `task.view_all` — see the "All Tasks" filter.

Lifecycle

not_started → in_progress → submitted → completed
 ■ cancelled
 ■ overdue (set automatically on deadline pass)

Common edits

- **Add a "labels" field** — column on `tasks` + migration, expose in the form, store, render.
- **Email when a task is reassigned.** Currently only when created. In `app/api/tasks/[id]/route.ts`, detect when `body.assigned_to` differs from the current row's, and call `sendTaskAssignedEmail` again.
- **Auto-mark overdue** — add a Vercel Cron job (see [EDITING-GUIDE Recipe 11](#)) that updates tasks with `deadline < now() AND status NOT IN ('completed', 'cancelled')` to `overdue` and sends `sendTaskOverdueEmail`.

3b. Task Requests (`/tasks/requests`)

What the user sees

- Anyone with `task.request` can submit a new task request from the "Requests" tab. The request goes into a pending queue — it is NOT a real task yet.
- Admin-level users (CEO / Owner / Co-Owner / Manager by default) get an inbox at `/tasks/requests` showing pending requests. They can **Approve** (creates the actual task) or **Deny** (records a required

reason).

- The requester can **Cancel** their own pending request.
- A red banner with the pending count appears on the dashboard / and on /tasks for approvers.

Manager scope

- CEO / Owner / Co-Owner see ALL pending requests.
- Manager sees only requests where the requester is one of their assigned agents (manager_assignments), plus requests explicitly targeted at them, plus their own.

Files

File	Role
database/migration-008.sql	Table + permission seeds.
app/tasks/requests/page.tsx	Server page — fetches the right rows per scope.
app/tasks/requests/TaskRequestsView.tsx	Client UI — list cards, filter tabs, new-request modal, deny modal.
app/api/task-requests/route.ts	GET (list, scope-aware) + POST (create).
app/api/task-requests/[id]/route.ts	PATCH with `action: 'approve'` \
app/tasks/page.tsx	Adds "Requests" tab + banner.
app/page.tsx	Adds dashboard banner.

Tables

- task_requests — main row, with status pending|approved|denied|cancelled, created_task_id link to the resulting task.
- tasks — written to on approve.
- task_updates — system "created from request" entry.
- notifications — bell-icon entries to approvers and to requester on resolution.
- audit_log — every create / approve / deny.

Permissions

- task.request — submit a request.
- task.approve_requests — see + approve/deny incoming requests.
- Manager scope is enforced in the API query, NOT in the permission system. Don't confuse "permission" (what your role can do at all) with "scope" (which rows it can do it on).

Common edits

- **Allow accountant to approve.** Sign in as CEO → /permissions → toggle on task.approve_requests for accountant. No code change.
- **Add a field to the request** (e.g. budget impact). Migration to add column → input in TaskRequestsView's new-request modal → accept in POST /api/task-requests → render in the card.

- **Auto-approve trivial requests.** In `POST /api/task-requests`, if the requester == assignee and priority != urgent, you could immediately create the task and skip the queue. Be careful — this defeats the audit-trail benefit.

For the full step-by-step record of how this was built, see [HOW-I-BUILT-task-requests.md](#).

4. Calendar (`/calendar`)

What the user sees

- Full-screen FullCalendar (month / week / day) with events colored by category, plus task deadlines as red blocks.
- Click an empty slot or "+ New Event" → create modal.
- Auto-pulled task deadlines are read-only (they live in `tasks` not `events`).

Files

File	Role
<code>app/calendar/page.tsx</code>	Server fetch of events ± 6 months around today, + tasks with deadlines.
<code>app/calendar/CalendarView.tsx</code>	Client component wrapping <code>@fullcalendar/react</code> . Holds the create modal.
<code>app/api/events/route.ts</code>	<code>POST</code> — create an event + attendees.

Tables

- `events`
- `event_attendees` (M:N junction)
- `tasks` (read-only — deadlines surface here)

Permissions

- `section.calendar`
- `event.create`
- `event.edit_any` / `event.delete` — currently no edit/delete API exposed.
- `event.invite_anyone` — invite others to events.

Common edits

- **Add recurring events.** The schema has a `recurring_pattern` column but it's not implemented. You'd need to expand it client-side (FullCalendar supports recurring via RRULE) and store the rule string.
- **Add event editing.** Add a `PATCH /api/events/[id]/route.ts` mirroring the create handler. Then add an edit modal in `CalendarView.tsx` triggered by clicking an event.
- **Change the date window** — `app/calendar/page.tsx` currently fetches ± 6 months. Change the math at the top of the file.

- **Add invitations by email.** After insert, loop over `attendees` and call a new `sendCalendarInviteEmail` in `lib/email.ts`.

5. Pipeline — Sales & Management (/pipeline/*)

This is the biggest, most complex module. Read it last.

Two pipelines, one table. The `pipeline_contacts` table has a `pipeline_type` column with two valid values:

- `sales` — debt-relief leads (consumers).
- `management` — B2B deals.

Each has its own page, its own status vocabulary, and its own set of UI labels — but they share the same code path.

What the user sees

- A list of contacts (cards).
- Click a contact → detail page with notes, call log, collaborators.
- "Add Contact" form at the top of the list.
- A built-in dialer for outbound calls (Twilio Voice SDK).

Files

File	Role
<code>app/pipeline/sales/page.tsx</code>	Sales list. Renders <code><PipelinePageBase></code> with <code>pipeline_type="sales"</code> .
<code>app/pipeline/management/page.tsx</code>	Management list. Same wrapper, different type.
<code>app/pipeline/sales/[id]/page.tsx</code> , <code>.../management/[id]/page.tsx</code>	Detail wrappers — both call <code><ContactDetailBase></code> .
<code>app/pipeline/PipelinePageBase.tsx</code>	The shared list page (server).
<code>app/pipeline/PipelineList.tsx</code>	The shared client list (cards, filters, modals).
<code>app/pipeline/ContactDetailBase.tsx</code>	Detail page (server).
<code>app/pipeline/ContactDetail.tsx</code>	Detail UI (client). Holds notes editor, call buttons, collaborator manager.
<code>app/api/pipeline/route.ts</code>	POST — create contact.
<code>app/api/pipeline/[id]/route.ts</code>	PATCH / DELETE.
<code>app/api/pipeline/[id]/calls/route.ts</code>	POST — manual call log entry.
<code>app/api/pipeline/[id]/notes/route.ts</code>	POST — add a sticky note.
<code>app/api/pipeline/notes/[id]/route.ts</code>	PATCH / DELETE — edit / delete a note.

File	Role
app/api/pipeline/[id]/collaborators/route.ts	GET, POST — list / add collaborators.
app/api/pipeline/collaborators/[collabId]/route.ts	DELETE.

Tables

- `pipeline_contacts` — both pipelines.
- `call_logs` — every outbound + inbound attempt.
- `contact_notes` — sticky notes (separate from per-call notes).
- `pipeline_collaborators` — additional users with access beyond `assigned_to`.

Permissions

- `section.pipeline_sales`, `section.pipeline_management`.
- `pipeline.create` — add contacts.
- `pipeline.edit_any` — edit contacts you aren't owner/collaborator on.
- `pipeline.delete` — delete contacts.
- `pipeline.view_all` — see contacts not assigned to you. (Without this, you only see your own + collaborator-assigned.)
- `pipeline.assign_to_anyone` — change `assigned_to` on a contact.
- `pipeline.export` — export to CSV (not implemented yet, but the flag exists).
- `call.log` — manually add a call log row.
- `call.make` — get a Twilio Voice token to actually call.
- `call.view_all` — see other people's call logs.

Owner / Collaborator model A contact has one `assigned_to`. Other users can be added as either:

- `collaborator` (role text in `pipeline_collaborators`) — full edit access.
- `observer` — read-only.

Many handlers do: `const canAccess = assigned_to === user.id || isCollaborator || hasPermission('pipeline.view_all')`.

Common edits

- **Add a new column** (e.g. "lead source"). Migration → form input in `PipelineList.tsx` → accept it in the POST handler → render it in `ContactDetail.tsx`.
- **Add a new status value.** Status is currently free text per pipeline type. Look in `app/pipeline/PipelineList.tsx` for the dropdown options array and add yours.
- **Export contacts as CSV.** New route `app/api/pipeline/export/route.ts` that gates on `pipeline.export`, queries `pipeline_contacts`, returns `text/csv` with `Content-Disposition: attachment`.

6. Phone calls — outbound + inbound (Twilio)

The most external-facing module. Lots of moving parts. Here's the whole picture.

6a. Outbound (browser dialer)

What happens

1. User clicks "Call" on a contact.
2. The browser asks `/api/twilio/voice-token` for a short-lived JWT scoped to the Voice SDK.
3. With that token, the SDK opens a peer connection.
4. Twilio bridges the SDK to the contact's phone number via `/api/twilio/twiml/dial`.
5. While the call runs, lifecycle webhooks (`/status`, `/recording`, `/amd`) hit our server with updates.
6. When the call ends, the browser POSTs to `/api/twilio/log-browser-call` so we persist the row.

Files

File	Role
<code>components/BrowserDialer.tsx</code>	The floating dial widget (mounted via pipeline + tasks).
<code>app/api/twilio/voice-token/route.ts</code>	Mints the Voice JWT.
<code>app/api/twilio/twiml/dial/route.ts</code>	Returns TwiML that bridges to the real phone number.
<code>app/api/twilio/status/route.ts</code>	Lifecycle callback — updates <code>call_logs</code> row.
<code>app/api/twilio/recording/route.ts</code>	Stores recording URL after it's ready.
<code>app/api/twilio/amd/route.ts</code>	Stores answering-machine detection result.
<code>app/api/twilio/log-browser-call/route.ts</code>	Persists the call row when browser side ends.
<code>lib/twilio.ts</code>	REST client + helpers.
<code>lib/twilio-voice.ts</code>	JWT minting for the Voice SDK.

6b. Inbound

What happens

1. Someone dials a Twilio number you own.
2. Twilio calls `/api/twilio/inbound`.
3. The handler reads `phone_number_routing` for that number and decides:
 - `agent` → ring a specific user's browser (TwiML `<Dial><Client>...`).
 - `available` → ring ALL online agents at once.
 - `voicemail` → straight to voicemail (records to `/api/twilio/voicemail`).
4. If a browser is online, `InboundCallListener` (mounted from Sidebar) shows the toast and bridges.

Files

File	Role
app/api/twilio/inbound/route.ts	Primary handler — decides routing.
app/api/twilio/inbound-fallback/route.ts	Used if /inbound errors.
app/api/twilio/voicemail/route.ts	Stores the voicemail recording.
app/api/twilio/voice-twiml/route.ts	Twiml for the browser identity.
components/InboundCallListener.tsx	Listens on the SDK for incoming calls; shows a toast.
app/twilio-numbers/... + /api/twilio/routing/...	Admin UI to assign routing per number.

Tables

- `call_logs` — every inbound + outbound attempt. `direction` = `inbound` / `outbound`. For inbound that didn't match a contact, `contact_id` is null and `from_number` carries the caller.
- `phone_number_routing` — admin config per Twilio number.
- `profiles.status` — used by "available" routing to pick agents.

Permissions

- `call.make` — required for the Voice token.
- `call.log` — manually log a call.
- `call.view_all` — see other people's call rows.

Webhook safety

- Twilio webhook paths are listed in `middleware.ts` (`TWILIO_WEBHOOK_PREFIXES`) so they bypass the cookie check.
- **In production, you should validate the `X-Twilio-Signature` header** before trusting the body. Currently not enforced — there's a TODO comment in `middleware.ts`. To add: in each webhook handler, import `validateRequest` from `twilio` and compare.

Common edits

- **Add a new outcome value** (e.g. `transferred`). Migration to extend the check (`outcome in (...)`) constraint on `call_logs`. Then update the dropdowns in `ContactDetail.tsx`.
- **Add SMS**. Currently no SMS. You'd add a route `POST /api/twilio/sms/send` that uses `twilio.messages.create(...)` and a webhook `/api/twilio/sms/inbound` (whitelist in `middleware`).
- **Change call recording behavior**. Twiml in `/api/twilio/twiml/dial/route.ts` has a `record="record-from-answer"` attribute. Remove or change to control recording.

7. Budget — Income, Expenses, Allocations (/budget)

What the user sees

- Dashboard at `/budget` — current month at-a-glance: planned vs actual per category, totals, recent expenses, recent income.
- `/budget/allocations` — set the monthly budget per category.
- Expense submission form (anywhere with the right permission).
- Expense list with approve/reject buttons (for approvers).

Files

File	Role
<code>app/budget/page.tsx</code>	Dashboard data fetch.
<code>app/budget/BudgetDashboard.tsx</code>	UI.
<code>app/budget/allocations/page.tsx</code> + <code>BudgetAllocations.tsx</code>	Per-category allocation editor.
<code>app/api/budgets/route.ts</code>	POST allocation.
<code>app/api/budgets/[id]/route.ts</code>	DELETE allocation.
<code>app/api/budgets/copy-previous/route.ts</code>	Copy last month's allocations forward.
<code>app/api/expenses/route.ts</code>	POST submit expense.
<code>app/api/expenses/[id]/route.ts</code>	PATCH approve/reject/edit.
<code>app/api/income/route.ts</code>	POST record income.

Tables

- `budgets` (one row per month×category; unique on that pair).
- `expenses` — `submitted` → `approved` → `reimbursed` (or `rejected`).
- `income`.

Permissions

- `section.budget`.
- `budget.view`.
- `budget.edit_allocations`.
- `expense.submit`.
- `expense.approve`.
- `income.record`.

Common edits

- **Add a new expense category.** Category is free text. Look in `BudgetAllocations.tsx` for the dropdown options — add yours.
- **Require receipt for expenses over \$X.** In the submit handler, check `body.amount > 100 && !body.receipt_drive_id` → return 400.
- **Email finance team on approve.** After the PATCH that flips status to `approved`, send via `sendEmail()`.

8. Files & Google Drive (/files)

What the user sees

- "Connect Google Drive" button if they haven't yet → opens OAuth in a new window → returns and shows their personal folder.
- A simple file list with upload + delete.
- Each user's files live in their own Drive folder (linked on `profiles.google_drive_folder_id`).

Files

File	Role
<code>app/files/page.tsx</code> + <code>FilesView.tsx</code>	UI.
<code>app/api/google/auth/route.ts</code>	Builds the consent URL, redirects there.
<code>app/api/google/callback/route.ts</code>	Receives the code, exchanges for tokens, stores them on the profile. Public — Google calls it.
<code>app/api/files/upload/route.ts</code>	POST a multipart-form file → uploads to Drive.
<code>app/api/files/[id]/route.ts</code>	DELETE a file.
<code>lib/google-drive.ts</code>	OAuth helpers + Drive API wrapper.

Tables

- `profiles.google_drive_folder_id`, `google_access_token`, `google_refresh_token`, `google_token_expires_at`.

Permissions

- `section.files`.
- No granular permissions on upload/delete — Drive is per-user.

Common edits

- **Force a specific folder structure.** In `lib/google-drive.ts`, the function that creates a user's root folder also creates sub-folders. Add to that list.
- **Allow uploading to a shared folder** (not just personal). You'd add a new "scope" param to `/api/files/upload` and route the upload to the relevant Drive folder ID.

9. Performance & Manager Assignments (/performance)

What the user sees

- Per-agent metrics: tasks completed, hours logged, calls made, conversion %.
- Leaderboards.

- Managers see only their assigned agents. Leadership sees everyone.
- `/performance/assignments` — admin UI to assign agents to managers.

Files

File	Role
<code>app/performance/page.tsx</code> + <code>PerformanceView.tsx</code>	Main dashboard.
<code>app/performance/assignments/page.tsx</code> + <code>AssignmentsManager.tsx</code>	Admin to wire managers↔agents.
<code>app/api/manager-assignments/route.ts</code> , <code>.../[id]/route.ts</code>	CRUD for the join table.
<code>lib/performance.ts</code>	The KPI computation logic. Pure functions over data.

Tables

- `manager_assignments` — (`manager_id`, `agent_id`) rows.

Permissions

- `section.performance`.
- `performance.view_all` — leadership.
- `performance.view_team` — managers (their assigned agents).
- `performance.assign_managers` — leadership only.

Common edits

- **Add a new metric.** In `lib/performance.ts`, extend `computeMetricsForUser()` to return a new field. Then render it in `PerformanceView.tsx`.
- **Change the metric window.** The default is "last 30 days" — change the date filter in `lib/performance.ts`.

10. Calculators (`/calculators`)

What the user sees

- Two tools: DTI (Debt-to-Income) and Budget calculators.
- Each is a form with live computed results.
- Can save a scenario → stored in `calculations`.

Files

File	Role
<code>app/calculators/page.tsx</code> + <code>CalculatorsView.tsx</code>	Wrapper.

File	Role
app/calculators/DTICalculator.tsx, BudgetCalculator.tsx	The actual UIs.
app/api/calculations/route.ts, .../[id]/route.ts	Save/edit/delete saved scenarios.

Tables

- `calculations` — owned by `user_id`.

Permissions

- `section.calculators`.

Common edits

- **Add a third calculator** — copy `DTICalculator.tsx` as a starting point, add a tab in `CalculatorsView.tsx`.
- **Add a new field to a saved scenario.** Migration → form input → save it via the POST/PATCH → render it back.

11. Team (`/team`)

What the user sees

- A list of all users with their role, status, Twilio number, etc.
- "Invite User" form — sends them a temp password by email.
- Buttons to change role, deactivate, etc.

Files

File	Role
app/team/page.tsx + TeamManager.tsx	UI.
app/api/users/invite/route.ts	Creates the profile, hashes a temp password, emails it.
app/api/users/[id]/route.ts	PATCH to change role / active.
app/api/users/[id]/twilio-number/route.ts	Assign a Twilio number.
lib/email.ts → <code>sendInviteEmail</code>	The invite email template.

Tables

- `profiles`.
- `audit_log` — every invite/role change.

Permissions

- `section.team`.
- `team.invite`.

- `team.change_roles`.
- `team.deactivate`.
- `team.delete` (currently no endpoint exposes hard delete — `is_active = false` is preferred).

Common edits

- **Pick a stronger temp password.** `app/api/users/invite/route.ts` — look for the `Math.random()` generation and increase length / use crypto.
- **Force password change on first login.** Add a `must_change_password` column to `profiles`, set it true on invite, and add a step in login that redirects to a "change password" page if true.

12. Custom Roles (`/roles`)

What the user sees

- A list of all roles (built-ins + custom).
- "New Role" button → create a custom role with key, label, color.
- Cannot delete built-ins.

Files

File	Role
<code>app/roles/page.tsx</code> + <code>RolesManager.tsx</code>	UI.
<code>app/api/roles/route.ts</code>	POST create.
<code>app/api/roles/[key]/route.ts</code>	PATCH rename/recolor, DELETE.

Tables

- `custom_roles` — registry of all roles. Built-ins have `is_built_in = true`.

Permissions

- `section.custom_roles`, `roles.manage`.

Common edits

- A custom role starts with **all permissions OFF**. Toggle them on at `/permissions`.
- Custom roles don't satisfy hard-coded helpers like `isLeadership()`. If you want a custom role to count as leadership, you'd need to update `lib/roles.ts` to read from the DB instead — non-trivial change.

13. Permissions Admin (`/permissions`)

What the user sees

- A giant matrix: rows = permission keys (grouped), columns = roles, cells = checkbox.

- Save → bulk POST `/api/permissions`.

Files

File	Role
<code>app/permissions/page.tsx</code> + <code>PermissionsManager.tsx</code>	UI.
<code>app/api/permissions/route.ts</code>	Bulk update. Calls <code>invalidatePermissionCache()</code> after.
<code>lib/permissions.ts</code>	Key list, labels, groups, cache.

Tables

- `role_permissions` — one row per (role, key).

Permissions

- `permissions.manage`.

Common edits

- See [EDITING-GUIDE § 7.4](#) for adding a new permission key.

14. Twilio Numbers Admin (`/twilio-numbers`)

What the user sees

- Each Twilio number you own with its current routing target.
- Edit routing per number: ring an agent, ring all available, or voicemail.

Files

File	Role
<code>app/twilio-numbers/page.tsx</code> + <code>TwilioNumbersManager.tsx</code>	UI.
<code>app/api/twilio/routing/route.ts</code>	Update routing.
<code>app/api/twilio/diagnostics/route.ts</code>	Sanity-check that env vars are set.

Tables

- `phone_number_routing` — `{ phone_number, routing_type, target_user_id }`.

Permissions

- `section.twilio_numbers`, leadership only.

15. Notifications (/notifications)

What the user sees

- A list of in-app notifications (bell-icon stream).
- Click a notification → marks read + navigates to its `Link`.

Files

- `app/notifications/page.tsx` — list + mark-read UI.
- The bell icon dropdown lives in `components/TopBar.tsx`.

Tables

- `notifications` — `is_read` flag, indexed on (`user_id`, `is_read`).

Permissions

- Available to everyone signed in.

Common edits

- **Add a new notification type.** Just `insert` into `notifications` from wherever your event happens — see [EDITING-GUIDE Recipe 6](#).

16. Settings (/settings)

What the user sees

- Their profile fields (name, email, phone, avatar).
- Change-password form (requires current password).
- Notification preferences (toggle email categories on/off).
- Disconnect Google Drive button.

Files

File	Role
<code>app/settings/page.tsx</code>	Server fetch.
<code>app/settings/SettingsForm.tsx</code>	Client UI.
<code>app/api/users/me/route.ts</code>	PATCH profile fields.
<code>app/api/users/me/password/route.ts</code>	PATCH password (compares current, sets new).
<code>app/api/users/me/google/disconnect/route.ts</code>	Clears Google tokens.

Tables

- `profiles`.

Common edits

- **Add a new preference toggle.** Edit the JSON shape stored in `profiles.notification_preferences` (it's a `jsonb` column — no migration needed for new keys). Show the toggle in `SettingsForm.tsx`. Read it in any email-sending code that should respect it.

17. Status & Presence (in TopBar)

What the user sees

- A status dropdown next to their avatar: Online / On The Line / In Meeting / On Break / Offline.
- A small colored dot on every team member's name elsewhere (synced from this).

Files

File	Role
<code>components/TopBar.tsx</code>	The dropdown UI. Marks "online" once per session.
<code>app/api/me/status/route.ts</code>	GET returns the whole team list with statuses. PATCH/POST updates mine.

Tables

- `profiles.status`, `status_message`, `status_updated_at`.

Permissions

- Available to everyone signed in.

Common edits

- **Add a new status value** — extend the `STATUSES` array in `TopBar.tsx` and add the new value to the DB CHECK constraint via a migration.
- **Auto-offline after N minutes of inactivity.** Today, status persists until manually changed. To add this: a Vercel Cron that runs `update profiles set status='offline' where status_updated_at < now() - interval '15 minutes'`.

PART 4

Recipes — Copy-Paste Templates

Source: docs/recipes.md

Recipes — Copy-Paste Templates

A library of solved problems. Find the recipe that matches your task, copy it, and adapt.

Each recipe is **self-contained** — read it without context and you can do it.

Index

Database

- [R1 — Add a column to an existing table](#)
- [R2 — Add a new table](#)
- [R3 — Add a foreign key](#)
- [R4 — Add an enum value to a CHECK constraint](#)
- [R5 — Add an index for a slow query](#)
- [R6 — Backfill data](#)

API routes

- [R7 — Add a GET endpoint](#)
- [R8 — Add a POST endpoint](#)
- [R9 — Add a PATCH endpoint](#)
- [R10 — Add a DELETE endpoint](#)
- [R11 — Validate request body with Zod](#)
- [R12 — Return CSV instead of JSON](#)
- [R13 — Upload a file \(multipart\)](#)

Pages

- [R14 — Add a new page](#)
- [R15 — Add a page with a dynamic ID](#)
- [R16 — Pass query-string filters to a page](#)
- [R17 — Add a modal form](#)
- [R18 — Refresh server data after a client action](#)

Permissions

- [R19 — Gate a page by permission](#)
- [R20 — Gate an API by permission](#)
- [R21 — "Owner or admin" check](#)
- [R22 — Hide a UI element by permission](#)

- [R23 — Add a new permission key](#)

Email & notifications

- [R24 — Send a templated email](#)
- [R25 — Create an in-app notification](#)
- [R26 — Respect user notification preferences](#)

Misc

- [R27 — Add a sidebar link](#)
- [R28 — Write an audit log entry](#)
- [R29 — Cache a permission check on the client](#)
- [R30 — Add a Twilio webhook](#)

R1 — Add a column to an existing table

```
-- database/migration-NNN.sql (NNN = next number)
alter table tasks add column if not exists priority_note text;
```

Run in Supabase SQL editor. Update the `.select()` and `.insert()` calls in your routes to include the new column.

R2 — Add a new table

```
-- database/migration-NNN.sql
create table if not exists projects (
  id uuid default uuid_generate_v4() primary key,
  name text not null,
  status text default 'active' check (status in ('active', 'archived')),
  owner_id uuid references profiles(id) not null,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

create index if not exists projects_owner_id_idx on projects(owner_id);
create index if not exists projects_status_idx on projects(status);

create trigger update_projects_updated_at before update on projects
for each row execute function update_updated_at_column();
```

R3 — Add a foreign key

```
-- Adding it via a new column:
alter table tasks add column if not exists project_id uuid references projects(id);
create index if not exists tasks_project_id_idx on tasks(project_id);
```

Use on delete cascade if child rows should die with the parent. Omit it for "created_by"-style references — you don't want deleting a user to nuke all their work.

R4 — Add an enum value to a CHECK constraint

Constraints can't be modified in place — drop and recreate.

```
alter table tasks drop constraint if exists tasks_status_check;
alter table tasks add constraint tasks_status_check
    check (status in ('not_started', 'in_progress', 'submitted', 'completed', 'overdue', 'cancelled', 'on_hold'));
```

R5 — Add an index for a slow query

Find the columns the query filters on (`WHERE` / `.eq()`) and the column it orders by.

```
-- Single column
create index if not exists tasks_assigned_to_idx on tasks(assigned_to);

-- Composite (when you filter on both columns together)
create index if not exists tasks_assigned_status_idx on tasks(assigned_to, status);

-- Partial (only useful subset of rows)
create index if not exists tasks_open_deadline_idx
    on tasks(deadline) where status not in ('completed', 'cancelled');
```

Always profile before adding indexes — they cost write performance.

R6 — Backfill data

When you add a column with a NOT NULL constraint, you need a default OR a backfill.

```
-- migration-NNN.sql
alter table tasks add column if not exists category text;

update tasks set category = 'general' where category is null;

alter table tasks alter column category set not null;
```

R7 — Add a GET endpoint

```
// app/api/projects/route.ts
import { NextResponse } from 'next/server';
import { getCurrentUser } from '@lib/auth';
import { supabaseAdmin } from '@lib/supabase';

export async function GET() {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const { data, error } = await supabaseAdmin
    .from('projects')
    .select('id, name, status')
    .eq('owner_id', user.id)
    .order('created_at', { ascending: false })
    .limit(100);

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json(data ?? []);
}
```

R8 — Add a POST endpoint

```
// app/api/projects/route.ts
export async function POST(request: Request) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  if (!(await hasPermission(user.role, 'project.create'))) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const body = await request.json();
  if (!body.name) return NextResponse.json({ error: 'Name required' }, { status: 400 });

  const { data, error } = await supabaseAdmin
    .from('projects')
    .insert({ name: body.name, owner_id: user.id })
    .select()
    .single();

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json(data);
}
```

R9 — Add a PATCH endpoint

```
// app/api/projects/[id]/route.ts
import { NextResponse } from 'next/server';
import { getCurrentUser } from '@lib/auth';
import { supabaseAdmin } from '@lib/supabase';

export async function PATCH(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const { id } = await params;
  const body = await request.json();

  // Only allow specific fields to be updated – never spread `...body`.
  const updates: any = { updated_at: new Date().toISOString() };
  if (typeof body.name === 'string') updates.name = body.name;
  if (typeof body.status === 'string') updates.status = body.status;

  const { data, error } = await supabaseAdmin
    .from('projects')
    .update(updates)
    .eq('id', id)
    .select()
    .single();

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json(data);
}
```

R10 — Add a DELETE endpoint

```
// app/api/projects/[id]/route.ts
export async function DELETE(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const { id } = await params;

  // Owner-or-admin check
  const { data: row } = await supabaseAdmin
    .from('projects').select('owner_id').eq('id', id).maybeSingle();
  if (!row) return NextResponse.json({ error: 'Not found' }, { status: 404 });

  const isOwner = row.owner_id === user.id;
  const isAdmin = await hasPermission(user.role, 'project.delete');
  if (!isOwner && !isAdmin) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const { error } = await supabaseAdmin.from('projects').delete().eq('id', id);
  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json({ ok: true });
}
```

R11 — Validate request body with Zod

```
import { z } from 'zod';

const Body = z.object({
  name: z.string().min(1).max(120),
  status: z.enum(['active', 'archived']).optional(),
  priority: z.number().int().min(1).max(5).optional(),
});

const parsed = Body.safeParse(await request.json());
if (!parsed.success) {
  return NextResponse.json({ error: parsed.error.issues }, { status: 400 });
}
const body = parsed.data;    // ← strictly typed now
```

R12 — Return CSV instead of JSON

```
const { data: rows } = await supabaseAdmin
  .from('pipeline_contacts').select('first_name, last_name, phone, status');

const header = ['First', 'Last', 'Phone', 'Status'];
const csv = [
  header.join(','),
  ...(rows ?? []).map(r =>
    [r.first_name, r.last_name, r.phone, r.status]
      .map(v => `"${v ?? ''}".replace(/"/g, '""')}`)
      .join(',')
  ),
].join('\n');

return new Response(csv, {
  headers: {
    'Content-Type': 'text/csv; charset=utf-8',
    'Content-Disposition': 'attachment; filename="contacts.csv"',
  },
});
```

R13 — Upload a file (multipart)

```
// app/api/files/upload/route.ts (server)
export async function POST(request: Request) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const formData = await request.formData();
  const file = formData.get('file') as File | null;
  if (!file) return NextResponse.json({ error: 'No file' }, { status: 400 });

  const bytes = Buffer.from(await file.arrayBuffer());
  // ... pass `bytes`, `file.name`, `file.type` to your storage helper
  return NextResponse.json({ ok: true });
}

// client side
const fd = new FormData();
fd.append('file', selectedFile);
await fetch('/api/files/upload', { method: 'POST', body: fd });
// note: do NOT set Content-Type – the browser sets it with the boundary
```

R14 — Add a new page

```
// app/projects/page.tsx
import { redirect } from 'next/navigation';
import { getCurrentUser } from '@lib/auth';
import { supabaseAdmin } from '@lib/supabase';
import Sidebar from '@components/Sidebar';
import TopBar from '@components/TopBar';

export default async function ProjectsPage() {
  const user = await getCurrentUser();
  if (!user) redirect('/login');

  const { data: projects } = await supabaseAdmin
    .from('projects').select('id, name').limit(100);

  return (
    <div className="flex min-h-screen bg-gray-50">
      <Sidebar user={user} />
      <main className="flex-1 min-w-0">
        <TopBar user={user} title="Projects" />
        <div className="p-4 sm:p-6 max-w-5xl mx-auto">
          {(projects ?? []).map(p => <div key={p.id}>{p.name}</div>)}
        </div>
      </main>
    </div>
  );
}
```

R15 — Add a page with a dynamic ID

```
// app/projects/[id]/page.tsx
import { notFound, redirect } from 'next/navigation';

export default async function ProjectPage(
  { params }: { params: Promise<{ id: string }> }
) {
  const user = await getCurrentUser();
  if (!user) redirect('/login');

  const { id } = await params;
  const { data: project } = await supabaseAdmin
    .from('projects').select('*').eq('id', id).maybeSingle();
  if (!project) notFound();

  return /* JSX */;
}
```

R16 — Pass query-string filters to a page

```
// app/projects/page.tsx
export default async function ProjectsPage(
  { searchParams }: { searchParams: Promise<{ status?: string; page?: string }> }
) {
  const params = await searchParams;
  const status = params.status || 'active';
  const page = parseInt(params.page || '0', 10);

  let q = supabaseAdmin.from('projects').select('*').eq('status', status);
  q = q.range(page * 25, page * 25 + 24);
  const { data } = await q;
  // ...
}
```

R17 — Add a modal form

The pattern is: a client component that holds an `open` boolean.

```
"use client";
import { useState } from 'react';

export default function NewProjectButton() {
  const [open, setOpen] = useState(false);
  const [name, setName] = useState('');

  async function submit(e: React.FormEvent) {
    e.preventDefault();
    const r = await fetch('/api/projects', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name }),
    });
    if (r.ok) { setOpen(false); window.location.reload(); }
  }

  return (
    <>
      <button type="button" className="btn-primary" onClick={() => setOpen(true)}>New Project</button>
      {open && (
        <div className="fixed inset-0 bg-black/50 z-50 flex items-center justify-center">
          <div className="bg-white rounded p-6 max-w-md w-full">
            <form onSubmit={submit}>
              <input value={name} onChange={e => setName(e.target.value)} className="input" />
              <div className="flex justify-end gap-2 mt-4">
                <button type="button" onClick={() => setOpen(false)}>Cancel</button>
                <button type="submit" className="btn-primary">Save</button>
              </div>
            </form>
          </div>
        </div>
      )}
    </>
  )
}
```

```
    );  
  }
```

R18 — Refresh server data after a client action

```
"use client";  
import { useRouter } from 'next/navigation';  
  
const router = useRouter();  
// ... after a successful fetch ...  
router.refresh(); // re-runs the server component on the current URL
```

For a full reload (rare): `window.location.reload()`.

R19 — Gate a page by permission

```
import { getCurrentUser } from '@lib/auth';  
import { hasPermission } from '@lib/permissions';  
import { redirect } from 'next/navigation';  
  
const user = await getCurrentUser();  
if (!user) redirect('/login');  
  
if (!(await hasPermission(user.role, 'section.projects'))) redirect('/');
```

R20 — Gate an API by permission

```
const user = await getCurrentUser();  
if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });  
  
if (!(await hasPermission(user.role, 'project.create'))) {  
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 });  
}
```

R21 — "Owner or admin" check

```
const { data: row } = await supabaseAdmin
  .from('projects').select('owner_id').eq('id', id).maybeSingle();
if (!row) return NextResponse.json({ error: 'Not found' }, { status: 404 });

const isOwner = row.owner_id === user.id;
const isAdmin = await hasPermission(user.role, 'project.edit_any');
if (!isOwner && !isAdmin) {
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
}
```

R22 — Hide a UI element by permission

In a server component, just `await hasPermission(user.role, key)`:

```
const canDelete = await hasPermission(user.role, 'project.delete');
return <DeleteButton show={canDelete} />;
```

In a client component, `fetch /api/me/permissions` once and check the map:

```
"use client";
const [perms, setPerms] = useState<Record<string, boolean>>({});
useEffect(() => {
  fetch('/api/me/permissions').then(r => r.json()).then(d => setPerms(d.permissions || {}));
}, []);
// ... {perms['project.delete'] && <button>Delete</button>}
```

But almost always, prefer doing this in the server component and passing the result as a prop.

R23 — Add a new permission key

Full walkthrough in [EDITING-GUIDE § 7.4](#). Quick version:

1. Add to `PERMISSION_KEYS` in `lib/permissions.ts`.
2. Add to `PERMISSION_LABELS` and `PERMISSION_GROUPS` in same file.
3. Seed in a migration:

```
insert into role_permissions(role, permission_key, enabled)
select unnest(array['ceo', 'owner', 'co-owner']), 'project.create', true
on conflict do nothing;
```

4. Use `hasPermission(user.role, 'project.create')` in your code.
-

R24 — Send a templated email

In `lib/email.ts`, add:

```
export async function sendProjectCreatedEmail(to: string, name: string, project: any) {
  const html = emailWrapper(
    `

## Hi ${name.split(' ')[0]},</h2> <p>A new project was created: <strong>${project.name}</strong>.</p>`, 'View Project', `${APP_URL}/projects/${project.id}`, ); return sendEmail(to, `New project: ${project.name}`, html); }


```

Then call it from your route:

```
await sendProjectCreatedEmail(targetUser.email, targetUser.full_name, data);
```

R25 — Create an in-app notification

```
await supabaseAdmin.from('notifications').insert({
  user_id: targetUserId,
  type: 'project_created',
  title: 'New project created',
  message: `${user.full_name} created "${project.name}"`,
  link: `/projects/${project.id}`,
});
```

R26 — Respect user notification preferences

`profiles.notification_preferences` is a `jsonb` blob. Read it before sending:

```
const { data: target } = await supabaseAdmin
  .from('profiles')
  .select('email, full_name, notification_preferences')
  .eq('id', targetId)
  .single();

if (target && target.notification_preferences?.task_assigned !== false) {
  // default to ON – only skip when explicitly false
  await sendTaskAssignedEmail(target.email, target.full_name, task);
}
```

To add a new preference key — just write it. The `jsonb` column doesn't need a migration for new keys.

R27 — Add a sidebar link

In `components/Sidebar.tsx`, find the group you want and add an item:

```
import { FolderKanban } from 'lucide-react';
// inside `groups`:
{
  label: 'Tools',
  items: [
    /* existing */
    {
      href: '/projects',
      label: 'Projects',
      icon: FolderKanban,
      show: can('section.projects', false),
    },
  ],
},
```

The `show` field controls visibility. `can('key', fallback)` checks the granular permission map; the fallback is what to show before perms load (use `true` for things everyone gets, `false` for restricted things).

R28 — Write an audit log entry

```
await supabaseAdmin.from('audit_log').insert({
  user_id: user.id,
  action: 'project_deleted',
  resource_type: 'project',
  resource_id: projectId,
  details: { name: project.name },
});
```

Do this for: create, delete, role changes, money movement, permission changes. Skip it for reads.

R29 — Cache a permission check on the client

The Sidebar already does this — it stores `/api/me/permissions` in `sessionStorage` for 5 minutes. To use the same cache elsewhere:

```
"use client";
import { useEffect, useState } from 'react';

function useMyPerms() {
  const [perms, setPerms] = useState<Record<string, boolean>>({});
  useEffect(() => {
    const cached = sessionStorage.getItem('perms_my');
    if (cached) {
      const { permissions, savedAt } = JSON.parse(cached);
      if (Date.now() - savedAt < 300_000) { setPerms(permissions); return; }
    }
  });
}
```

```

    fetch('/api/me/permissions').then(r => r.json()).then(d => {
      setPerms(d.permissions || {});
      sessionStorage.setItem('perms_my', JSON.stringify({
        permissions: d.permissions || {}, savedAt: Date.now(),
      }));
    });
  }, []);
  return perms;
}

```

R30 — Add a Twilio webhook

Twilio webhooks are POSTed by Twilio's servers, so they bypass the cookie check.

1. Create the route, e.g. `app/api/twilio/recording-status/route.ts`:

```

import { NextResponse } from 'next/server';
export async function POST(request: Request) {
  const form = await request.formData();
  const callSid = form.get('CallSid');
  // ... do stuff
  return new Response('', { status: 200 });
}

```

2. Whitelist the path in `middleware.ts`:

```

const TWILIO_WEBHOOK_PREFIXES = [
  // existing
  '/api/twilio/recording-status',
];

```

3. Configure the URL in the Twilio console for whatever event you're handling.

4. **In production**, validate X-Twilio-Signature:

```

import twilio from 'twilio';
const url = `${process.env.NEXT_PUBLIC_APP_URL}${request.nextUrl.pathname}`;
const sig = request.headers.get('x-twilio-signature') || '';
const params = Object.fromEntries(form.entries());
const ok = twilio.validateRequest(process.env.TWILIO_AUTH_TOKEN!, sig, url, params);
if (!ok) return new Response('Forbidden', { status: 403 });

```

PART 5

File Index

Source: docs/file-index.md

File Index — Every File Mapped to Its Purpose

When you don't know which file to edit, search this page first.

Top-level config

File	What it does	Edit when
<code>package.json</code>	Dependencies + npm scripts	Adding/removing a library
<code>next.config.js</code>	Next.js build options (TS-ignore, image domains, headers, bundle optimizations)	Tuning build behavior
<code>tailwind.config.js</code>	Tailwind CSS palette + font setup	Adding a brand color, changing fonts
<code>postcss.config.js</code>	PostCSS pipeline for Tailwind	Almost never
<code>tsconfig.json</code>	TypeScript compiler options	Almost never
<code>middleware.ts</code>	The cookie gate — runs before every request	Whitelisting a new public path (Twilio webhook, cron, etc.)
<code>next-env.d.ts</code>	Next.js TS types	Never (auto-managed)
<code>.env.example</code>	Template for environment variables	Adding a new secret
<code>README.md</code>	Operator/setup guide	Changing install steps
<code>HOW-TO-APPLY.txt</code>	Quick deployment notes	Updating deploy procedure

app/ — pages + API routes

Root pages

Path	File	Notes
/	<code>app/page.tsx</code>	Dashboard. KPI cards + task list + today's events. Pure server component.
/login	<code>app/login/page.tsx</code>	The only un-authed page. Has a client form.

Path	File	Notes
Root layout	app/layout.tsx	<html><body> wrapper. Don't put auth here — pages handle it.
Global styles	app/globals.css	Tailwind base + brand custom CSS (.btn-primary, .card, etc).

Tasks (/tasks)

Path	File	Notes
List	app/tasks/page.tsx	Filter tabs by query string.
New	app/tasks/new/page.tsx + NewTaskForm.tsx	Server page + client form.
Detail	app/tasks/[id]/page.tsx	Joins task + updates + notes.
Detail tracker	app/tasks/[id]/TaskWorkTracker.tsx	Client component — status toggle + work log.

Calendar (/calendar)

Path	File	Notes
Page	app/calendar/page.tsx	Fetches ± 6 months.
UI	app/calendar/CalendarView.tsx	Client wrapper around FullCalendar.

Pipeline (/pipeline/*)

Path	File	Notes
Sales list	app/pipeline/sales/page.tsx	Calls <PipelinePageBase pipeline_type="sales">.
Mgmt list	app/pipeline/management/page.tsx	Same with "management".
Sales detail	app/pipeline/sales/[id]/page.tsx	Calls <ContactDetailBase>.
Mgmt detail	app/pipeline/management/[id]/page.tsx	Same.
Shared list page	app/pipeline/PipelinePageBase.tsx	Server.
Shared list UI	app/pipeline/PipelineList.tsx	Client. Holds add-contact form + filters.

Path	File	Notes
Shared detail page	app/pipeline/ContactDetailBase.tsx	Server.
Shared detail UI	app/pipeline/ContactDetail.tsx	Client. Notes, calls, collaborators.

Budget (/budget)

Path	File	Notes
Dashboard page	app/budget/page.tsx	Server fetch.
Dashboard UI	app/budget/BudgetDashboard.tsx	Client. Charts + lists.
Allocations page	app/budget/allocations/page.tsx	Server.
Allocations UI	app/budget/allocations/BudgetAllocations.tsx	Client. Per-month, per-category editor.

Calculators (/calculators)

Path	File	Notes
Page	app/calculators/page.tsx	Wrapper.
Tab switcher	app/calculators/CalculatorSView.tsx	Client.
DTI tool	app/calculators/DTICalculator.tsx	Client.
Budget tool	app/calculators/BudgetCalculator.tsx	Client.

Files (/files)

Path	File	Notes
Page	app/files/page.tsx	Checks if Drive is connected.
UI	app/files/FilesView.tsx	Client. Upload + list.

Team (/team)

Path	File	Notes
Page	app/team/page.tsx	Server.

Path	File	Notes
UI	app/team/TeamManager.tsx	Client. Invite + role-change controls.

Performance (/performance)

Path	File	Notes
Page	app/performance/page.tsx	Server. Routes to assigned-only for managers.
UI	app/performance/PerformanceView.tsx	Client. Leaderboards.
Assignments page	app/performance/assignments/page.tsx	Server.
Assignments UI	app/performance/assignments/AssignmentsManager.tsx	Client.

Settings (/settings)

Path	File	Notes
Page	app/settings/page.tsx	Server.
Form	app/settings/SettingsForm.tsx	Client. Profile + password + notifications.

Admin pages

Path	File	Notes
Permissions page	app/permissions/page.tsx	Server.
Permissions UI	app/permissions/PermissionsManager.tsx	Client matrix.
Roles page	app/roles/page.tsx	Server.
Roles UI	app/roles/RolesManager.tsx	Client.
Twilio numbers page	app/twilio-numbers/page.tsx	Server.
Twilio numbers UI	app/twilio-numbers/TwilioNumbersManager.tsx	Client.

Notifications

Path	File	Notes
Page	app/notifications/page.tsx	Server.

app/api/ — backend routes

Auth

Endpoint	File	Method	Notes
/api/auth/login	app/api/auth/login/route.ts	POST	Validates pwd, sets cookie.
/api/auth/logout	app/api/auth/logout/route.ts	POST	Clears cookie.
/api/auth/setup	app/api/auth/setup/route.ts	POST	First CEO only.

"Me" routes

Endpoint	File	Method
/api/me/permissions	app/api/me/permissions/route.ts	GET
/api/me/status	app/api/me/status/route.ts	GET / PATCH / POST
/api/users/me	app/api/users/me/route.ts	PATCH
/api/users/me/password	app/api/users/me/password/route.ts	PATCH
/api/users/me/google/disconnect	app/api/users/me/google/disconnect/route.ts	POST

Tasks

Endpoint	File	Method
/api/tasks	app/api/tasks/route.ts	POST
/api/tasks/[id]	app/api/tasks/[id]/route.ts	PATCH
/api/tasks/[id]/updates	app/api/tasks/[id]/updates/route.ts	POST

Endpoint	File	Method
/api/tasks/[id]/notes	app/api/tasks/[id]/notes/route.ts	POST

Events

Endpoint	File	Method
/api/events	app/api/events/route.ts	POST

Pipeline

Endpoint	File	Method
/api/pipeline	app/api/pipeline/route.ts	POST
/api/pipeline/[id]	app/api/pipeline/[id]/route.ts	PATCH, DELETE
/api/pipeline/[id]/calls	app/api/pipeline/[id]/calls/route.ts	POST
/api/pipeline/[id]/notes	app/api/pipeline/[id]/notes/route.ts	POST
/api/pipeline/notes/[id]	app/api/pipeline/notes/[id]/route.ts	PATCH, DELETE
/api/pipeline/[id]/collaborators	app/api/pipeline/[id]/collaborators/route.ts	GET, POST
/api/pipeline/collaborators/[collabId]	app/api/pipeline/collaborators/[collabId]/route.ts	DELETE

Budget

Endpoint	File	Method
/api/budgets	app/api/budgets/route.ts	POST
/api/budgets/[id]	app/api/budgets/[id]/route.ts	DELETE
/api/budgets/copy-previous	app/api/budgets/copy-previous/route.ts	POST
/api/expenses	app/api/expenses/route.ts	POST
/api/expenses/[id]	app/api/expenses/[id]/route.ts	PATCH
/api/income	app/api/income/route.ts	POST

Files & Google

Endpoint	File	Method
/api/google/auth	app/api/google/auth/route.ts	GET (redirect)
/api/google/callback	app/api/google/callback/route.ts	GET (Google calls)
/api/files/upload	app/api/files/upload/route.ts	POST
/api/files/[id]	app/api/files/[id]/route.ts	DELETE

Users / Team

Endpoint	File	Method
/api/users/invite	app/api/users/invite/route.ts	POST
/api/users/[id]	app/api/users/[id]/route.ts	PATCH
/api/users/[id]/twilio-number	app/api/users/[id]/twilio-number/route.ts	PATCH

Roles / Permissions admin

Endpoint	File	Method
/api/roles	app/api/roles/route.ts	POST
/api/roles/[key]	app/api/roles/[key]/route.ts	PATCH, DELETE
/api/permissions	app/api/permissions/route.ts	POST

Performance

Endpoint	File	Method
/api/manager-assignments	app/api/manager-assignments/route.ts	POST
/api/manager-assignments/[id]	app/api/manager-assignments/[id]/route.ts	DELETE

Calculators

Endpoint	File	Method
/api/calculations	app/api/calculations/route.ts	POST
/api/calculations/[id]	app/api/calculations/[id]/route.ts	PATCH, DELETE

Twilio — browser side

Endpoint	File	Method	Notes
/api/twilio/voice-token	app/api/twilio/voice-token/route.ts	GET	Mints Voice SDK JWT.
/api/twilio/call	app/api/twilio/call/route.ts	POST	Server-initiated call.
/api/twilio/diagnostics	app/api/twilio/diagnostics/route.ts	GET	Sanity-check env vars.
/api/twilio/log-browser-call	app/api/twilio/log-browser-call/route.ts	POST	After browser call ends.
/api/twilio/routing	app/api/twilio/routing/route.ts	POST	Set inbound routing for a number.
/api/twilio/recordings/[id]	app/api/twilio/recordings/[id]/route.ts	GET	Proxy a recording with auth.

Twilio — webhook side (public, called by Twilio)

Endpoint	File	Method	Notes
/api/twilio/twiml/dial	app/api/twilio/twiml/dial/route.ts	POST, GET	Twiml for outbound bridge.
/api/twilio/status	app/api/twilio/status/route.ts	POST	Call lifecycle.
/api/twilio/recording	app/api/twilio/recording/route.ts	POST	Recording ready.
/api/twilio/amd	app/api/twilio/amd/route.ts	POST	Answering-machine detection.
/api/twilio/voice-twiml	app/api/twilio/voice-twiml/route.ts	POST, GET	Browser identity Twiml.

Endpoint	File	Method	Notes
/api/twilio/inbound	app/api/twilio/inbound/route.ts	POST, GET	Primary inbound — selects target.
/api/twilio/inbound-fallback	app/api/twilio/inbound-fallback/route.ts	POST, GET	Fallback if /inbound errors.
/api/twilio/voicemail	app/api/twilio/voicemail/route.ts	POST	Voicemail recording callback.

All Twilio webhooks are whitelisted in `middleware.ts` → `TWILIO_WEBHOOK_PREFIXES`.

components/ — shared client components

File	Purpose
Sidebar.tsx	Left nav. Renders permission-gated nav items. Mounts <code>InboundCallListener</code> .
TopBar.tsx	Header. Title + bell + status dropdown + (presence).
BrowserDialer.tsx	Floating outbound-call widget. Embed it in any page that needs it.
InboundCallListener.tsx	Invisible. Connects to Twilio Voice SDK; pops a toast on inbound.

lib/ — server-side utilities

File	Has <code>server-only</code> ?	Purpose
auth.ts	Yes	<code>getCurrentUser</code> , JWT sign/verify, cookie helpers, password hash.
roles.ts	No — safe on client	Hard-coded role helpers like <code>canManageUsers()</code> .
permissions.ts	Yes	<code>hasPermission()</code> , <code>getPermissions()</code> , the key list, cache.
supabase.ts	No (but uses service key)	Exports <code>supabase</code> (anon) and <code>supabaseAdmin</code> (service).
email.ts	No (but reads <code>RESEND_API_KEY</code>)	All email templates + <code>sendEmail</code> .

File	Has <code>server-only</code> ?	Purpose
<code>google-drive.ts</code>	Yes	OAuth flow + Drive REST.
<code>twilio.ts</code>	Yes	Twilio REST client + helpers.
<code>twilio-voice.ts</code>	Yes	Voice SDK JWT minting.
<code>performance.ts</code>	Yes	KPI computations.

Rule: never import a `server-only` file from a `"use client"` component. If you need role checks in the browser, use `lib/roles.ts`.

database/ — SQL

File	Purpose	When to edit
<code>schema.sql</code>	Initial install snapshot	Almost never. Used when setting up a fresh Supabase project.
<code>migration-002.sql</code> through <code>migration-007.sql</code>	Past changes in order	Never edit — they were already run in production.
<code>migration-NNN.sql</code> (new)	Add new changes here	Every time you change the schema.

docs/ — you are here

File	Purpose
<code>README.md</code>	Doc index.
<code>EDITING-GUIDE.md</code>	The master "how to edit" doc. Start here.
<code>feature-walkthroughs.md</code>	Every feature explained.
<code>recipes.md</code>	Copy-paste templates.
<code>file-index.md</code>	This file.
<code>architecture.md</code>	High-level architecture.
<code>data-model.md</code>	DB tables and relationships.
<code>auth-and-permissions.md</code>	Auth & permission deep-dive.
<code>api-reference.md</code>	API at a glance.
<code>dev-workflow.md</code>	Running locally, common tasks.
<code>performance.md</code>	Perf notes — what was slow, what was fixed.

File	Purpose
troubleshooting.md	Error → cause → fix.
ui-and-styling.md	Tailwind setup, brand classes, component patterns.

PART 6

How I Built: Task Requests

Source: docs/HOW-I-BUILT-task-requests.md

How I Built the Task-Requests Feature

A complete, blow-by-blow record of building a real feature in this codebase. Use this as a reference next time you want to build something yourself — the order, the decisions, and the file map all transfer to any other feature.

What we built (recap)

A "Request a task" workflow:

- Anyone with `task.request` can submit a task they'd like created. It doesn't become a real task yet — it sits in a pending queue.
- Anyone with `task.approve_requests` (CEO / Owner / Co-Owner / Manager by default) gets an inbox of pending requests.
- **Manager scope:** a manager only sees requests from agents linked to them via `manager_assignments`. Leadership (CEO/Owner/Co-Owner) sees everything.
- Approving creates the real task (linked back to the request) and sends the normal assignment notification + email.
- Denying records a required reason and notifies the requester.
- The requester can cancel their own pending request.
- A red banner shows pending count on the dashboard and `/tasks` page.

The order I did everything (and why)

#	Step	Why this order
1	Decide the data shape	You can't write API/UI until you know what columns and rules exist.
2	Write the SQL migration	Run it against Supabase before any code talks to it.
3	Register the new permission keys	The migration seeds them; the TS code references them. Doing both together avoids drift.
4	Write the API routes	These are the contract the UI consumes. Write + test the backend first; the frontend is "just" a call to it.
5	Build the page + UI	Server page does the data fetching, the client component handles interactivity.

#	Step	Why this order
6	Integrate the surface (tabs, banner, sidebar)	The feature has to be discoverable. Wire it in last, after it works in isolation.
7	Type-check	Quick <code>npx tsc --noEmit</code> catches dumb mistakes.
8	Run the migration in Supabase	Only now — by this point the SQL has been read many times.

Most important rule: don't skip steps 1 or 2. I've never seen anyone get hurt by spending 10 extra minutes designing the data model. I've seen *lots* of pain caused by writing code before knowing what the columns will be.

Step 1 — Designing the data shape

Before opening any file, I wrote down the answers to these questions:

What is a "task request"? What does it know? What states can it be in?

A task request has:

- The task's *would-be* fields: title, description, priority, deadline, estimated_days.
- A requester (the person asking).
- An assignee (the person it would be assigned to — often = requester).
- Optionally, a specific admin the requester wants to review it.
- A status: `pending` → `approved` / `denied` / `cancelled`.
- A resolution record: who resolved it, when, why (if denied).
- A pointer back to the actual `tasks` row once approved.

Who can see what?

- Approvers see their inbox (scoped by role).
- Requesters see their own requests in any state.

What constraints?

- Status is an enum: `pending`, `approved`, `denied`, `cancelled`.
- Priority enum matches `tasks.priority` exactly so approval can copy fields 1:1.

That's all the design needed. **15 minutes of writing this down saved me from a dozen wrong commits.**

Step 2 — The SQL migration

File: `database/migration-008.sql`

I picked the next number after the highest existing migration (`migration-007.sql` → 008). The pattern in every migration is:

1. Comment block at the top explaining what and why.
2. `create table if not exists` (always `if not exists` so re-running is safe).
3. Indexes for every column the app will filter on.
4. `seed_permission(...)` calls for any new permission keys.

Choices and trade-offs

- **Used** `references profiles(id)` **without** `on delete cascade` for `requested_by` and `assigned_to`. Reason: if a user is deactivated, we don't want their request history to disappear.
- **Used** `on delete set null` on `created_task_id`. Reason: a task can be hard-deleted later, but we want the request history intact — just the link broken.
- **A partial index** on `(created_at desc) where status = 'pending'`. Reason: the approver inbox query is `where status='pending' order by created_at desc`. A partial index there is small and fast.
- `status` is **text with a CHECK constraint**, not a Postgres enum type. Reason: consistent with the rest of the codebase. Adding a new value is one `alter table` (per `data-model.md` § conventions).
- `seed_permission` is the helper from `migration-003.sql` — I reused it rather than writing `insert ... on conflict do nothing` ten times.

The seed defaults

```
-- task.request: most roles
ceo / owner / co-owner / manager / employee / accountant → true
viewer                                                    → false

-- task.approve_requests: admin only
ceo / owner / co-owner / manager → true
everyone else                    → false
```

Manager scope isn't a permission — it's a *query rule* enforced in the API. Permissions answer "can this role do X at all"; scope answers "on which rows." Don't confuse them.

Step 3 — Register the permission keys in TypeScript

File: `lib/permissions.ts`

Three additions:

1. `PERMISSION_KEYS` — add `'task.request'` and `'task.approve_requests'` to the const array.

2. `PERMISSION_GROUPS` — drop them in the existing "Tasks" group so they appear next to the related keys in the admin matrix.
3. `PERMISSION_LABELS` — friendly UI strings.

***Why is this in TS and in SQL?** The DB is the source of truth at runtime. The TS array is what makes the keys TYPE-SAFE in your code (autocomplete + compiler check) and what feeds the admin UI's matrix. Both lists must stay in sync — the seed function lives in the migration so SQL doesn't need to know the TS list at runtime, but the TS file needs to know all keys so the admin UI renders them all.*

This is the place I most often forget step 3 (the labels). Without labels the admin page still works, but the checkbox shows the raw key like `task.approve_requests`, which looks ugly.

Step 4 — API routes

I built two routes:

`app/api/task-requests/route.ts`

- `GET` — list, with scope enforcement.
- `POST` — create a request.

The scope logic in `GET` is the most interesting part. Read this top-to-bottom — it's the pattern for any "leadership sees all, manager sees their team, others see their own" rule:

```
const canApprove = await hasPermission(user.role, 'task.approve_requests');

if (mine || !canApprove) {
  // Not an approver, or they asked for "mine" – own requests only
  q = q.eq('requested_by', user.id);
} else if (!isLeadership(user.role)) {
  // Approver but not leadership → manager scope
  const { data: assigns } = await supabaseAdmin
    .from('manager_assignments').select('agent_id').eq('manager_id', user.id);
  const visibleRequesters = [user.id, ...((assigns ?? []).map(a => a.agent_id))];
  const inList = visibleRequesters.map(id => ` "${id}" `).join(',');
  q = q.or(`requested_by.in.${inList}`, target_approver.eq.${user.id}`);
}
// else leadership: no filter
```

Three things to notice:

1. **The `.or(...)` syntax is finicky.** Each clause is `column.operator.value`. For an `IN`, it's `column.in.(val1, val2, ...)`. Strings have to be double-quoted inside the parens.
2. **We compute `visibleRequesters` in JS** rather than try to express the manager-scope rule as one SQL query. It's two simple queries and reads way clearer.
3. **The `mine || !canApprove` short-circuit** handles the regular-user case naturally. If you're not an approver, you can **ONLY** see your own requests — same code path as an approver who clicked "My Requests".

```
app/api/task-requests/[id]/route.ts
```

- PATCH with an `action` field: `approve` / `deny` / `cancel`.

Three things this route does that are worth copying for similar features:

1. **Fetch the row first, then check state.** Refuse anything that isn't `pending`. This prevents double-approvals.
2. **The approve action does multiple writes:** create the task, link it back, write the system update on the task, send the assignment notification, notify the requester. I did NOT wrap these in a transaction — Supabase doesn't expose transactions to the JS client. If a later step fails, the earlier writes stay. For non-financial workflows like this, that's an acceptable trade. (For payments etc., use a Postgres function called via `rpc()`.)
3. **Manager-scope check is duplicated** here from the GET route. That's intentional — defense in depth. A manager could try to PATCH a request that wasn't in their list view. The check has to live in both places.

Step 5 — The UI

Split into two files following the standard pattern (server page + client component):

```
app/tasks/requests/page.tsx (server)
```

This page does the same scope logic as the GET API — both to render the right data and to keep things fast (no client round-trip to fetch the list on first paint).

A subtlety: the page also computes the **list of eligible approvers** for the "Send to" dropdown in the new-request modal. Eligibility = anyone whose role has `task.approve_requests` enabled. I did this by computing the granular permission for each *distinct role* (not each user) — much fewer queries.

```
app/tasks/requests/TaskRequestsView.tsx (client)
```

Holds the modals (new request, deny reason), the action handlers, the filter tabs.

The new-request modal is intentionally similar to the existing `NewTaskForm` so users don't have to learn a new layout — same fields, same priority dropdown, same deadline pattern. The only new field is "Send to (optional)" — leave it blank to broadcast to all eligible approvers, or pick someone specific.

Step 6 — Wiring it into the surface

A feature isn't done until people can find it. I wired it in three places:

1. **Tab on** `/tasks` — a "Requests" link with a red badge showing the pending count (only visible to approvers when there are pending items).

2. **Banner on /tasks** — a more prominent red call-to-action box right above the task grid when there's pending stuff to approve.
3. **Banner on the dashboard /** — same banner, so admins see it on first nav.

I deliberately did NOT add a separate sidebar entry. Reasons:

- The sidebar is already pretty long.
- Task requests are *part of* the tasks workflow — keeping it under `/tasks/*` and surfacing it from there preserves the mental model.
- The banner-with-count is more eye-catching than another nav item buried in a group.

If you disagree, adding it to the sidebar is one entry in `components/Sidebar.tsx` (see `recipes.md` → R27).

Step 7 — Type-check

```
npx tsc --noEmit
```

The only error introduced by my code was a Supabase join-type quirk:

`select('...assignee:profiles!fk(...)' )` is typed as `assignee: T[]` in the generated types, even though the FK is single-valued so the real return is `assignee: T`. I fixed it with a single `as any` cast at the prop boundary. The cleaner fix would be writing explicit row types, but for one place where the join shape is obvious, the cast is fine.

Pre-existing errors in `TeamManager.tsx`, `PermissionsManager.tsx`, etc. are unrelated to this work and don't block the build (`next.config.js` has `ignoreBuildErrors: true`).

Step 8 — Run the migration

The migration runs in Supabase, not by any automation in this repo.

1. Open Supabase Dashboard → SQL Editor.
2. Paste the entire contents of `database/migration-008.sql`.
3. Click **Run**.
4. Verify:

```
select * from task_requests;           -- should exist (empty)
select role, permission_key, enabled
  from role_permissions
 where permission_key in ('task.request', 'task.approve_requests')
 order by role, permission_key;
```

Until this runs, the feature returns 500s because the table doesn't exist. So this is a hard gate before users see anything.

Files touched (the full list)

database/migration-008.sql	NEW	schema + permission seeds
lib/permissions.ts	EDIT	add 2 keys + labels + group
app/api/task-requests/route.ts	NEW	GET (list) + POST (create)
app/api/task-requests/[id]/route.ts	NEW	PATCH (approve/deny/cancel)
app/tasks/requests/page.tsx	NEW	server page for the inbox
app/tasks/requests/TaskRequestsView.tsx	NEW	client UI: list + modals + actions
app/tasks/page.tsx	EDIT	add "Requests" tab + banner
app/page.tsx	EDIT	add dashboard banner

8 files, of which 5 are new and 3 are edits. That's a typical mid-size feature — for a really small feature you might only touch 3-4 files; for a big one (e.g. the pipeline), you'd touch 20.

Lessons I'd reapply to future features

What worked

- **Designed the data first.** Once the columns were on paper, every code question had an obvious answer.
- **Wrote API before UI.** I could mentally test the backend with curl before any UI existed — the UI became "the prettiest way to call these endpoints" instead of "the place where I figure out what I want."
- **Reused existing helpers** — `seed_permission` SQL function, `isLeadership()`, `sendTaskAssignedEmail()`. Even when I had to add a new feature, the surrounding glue was already there.
- **Defense in depth on permissions** — same checks at the page level AND the API level. Stops UI bugs from becoming security bugs.
- **Surfaced it in two places** (banner + tab). Discovery matters more than people give it credit for.

What I'd watch for

- **Supabase `.or()` syntax is fragile.** If you forget to quote a UUID, the parse silently fails and you get unexpected rows. Print the query string and eyeball it the first time.
- **No transactions.** If I added another step to the approve path (say, posting to Slack), and Slack failed after the task was created, the request would still be marked approved but with a side-effect failure. For higher-stakes flows, push it into a Postgres function and call via `supabaseAdmin.rpc()`.
- **The "any eligible admin" notification fan-out** is $O(\text{approvers})$ inserts into `notifications`. Fine at our scale. If we had hundreds of admins it'd be wasteful — a "queue" pattern with a single row + a join would be better.

The reusable template for "approval workflows"

If you ever want a request-then-approve flow for something else (expenses, time-off, role changes...), here's the pattern in one block:

1. Migration:
 - Table with: actor (who asked), target (what/who it affects), payload (the fields it would create), status (pending/approved/denied/cancelled), resolver (who acted on it), resolved_at, denial_reason, linked_resource_id.
 - Indexes on status, actor, and a partial index on pending.
 - Seed two permissions: <thing>.request and <thing>.approve_requests.
2. lib/permissions.ts:
 - Add the two keys, labels, and group.
3. app/api/<thing>-requests/route.ts:
 - GET with scope filter (leadership all / manager team / others self).
 - POST that inserts + notifies approvers.
4. app/api/<thing>-requests/[id]/route.ts:
 - PATCH with action: approve / deny / cancel.
 - approve creates the real resource + links it back.
 - deny requires a reason + notifies requester.
 - cancel only by requester, only while pending.
5. app/<thing>s/requests/page.tsx + RequestsView.tsx:
 - Same shape as task requests – filter tabs, cards, modals.
6. Surface it: tab on the parent page + banner on dashboard.
7. Type-check, run migration, test.

Every "approval" feature in any web app fits in those 7 steps. The only thing that changes is the payload shape.

PART 7

Architecture

Source: docs/architecture.md

Architecture

A single Next.js 14 (App Router) app that handles both the UI and the API. Everything that talks to the database, sends email, or calls Twilio runs on the server. The browser only sees rendered HTML + small client islands for interactivity.

Request lifecycle (page navigation)

Browser → middleware.ts → app/<route>/page.tsx (server) → Supabase → HTML → client islands hydrate

1. **Browser hits a URL.**
2. `middleware.ts` runs first. It checks for the `debtrex_session` cookie, verifies the JWT with `jose`, and either lets the request through or redirects to `/login`. Public paths (login API, Twilio webhooks, `/api/google/callback`) are allowed without a cookie.
3. **The page is a React Server Component.** It calls `getCurrentUser()` (validates JWT, fetches the profile row), then runs its data queries against Supabase using the **service-role client** (`supabaseAdmin`).
4. **Permission check.** Pages with restricted access call `hasPermission(role, key)`. The permissions table is loaded once and cached in process memory for 5 minutes.
5. **HTML streams back.** Client components ("`use client`") hydrate for interactivity — modals, dropdowns, optimistic state.

Request lifecycle (API call)

Browser fetch → middleware.ts → app/api/<route>/route.ts → `getCurrentUser` → permission check → supabase

API routes follow the exact same auth flow as pages. Every route handler should:

1. Call `getCurrentUser()` and return 401 if null.
2. Call `hasPermission(user.role, '<key>')` for the action. Return 403 if false.
3. Do the work. Return JSON.

Look at `app/api/tasks/route.ts` for the canonical shape.

Two Supabase clients (important)

```
// lib/supabase.ts
export const supabase          // anon key – for browser-safe reads. Almost never used in this app.
export const supabaseAdmin     // service-role key – bypasses RLS. Used everywhere on the server.
```

This app does **not** use Postgres RLS. Authorization is enforced entirely in TypeScript (via `hasPermission()` and the helpers in `lib/roles.ts`). That means:

- Every API route **MUST** do its own permission check. There's no DB safety net.
- The browser never gets the service-role key — it's a server-only secret.
- If you ever add code paths where the browser talks to Supabase directly, you need to switch on RLS or you'll have a hole.

Auth model — JWT cookies, not Supabase Auth

- Users live in our own `profiles` table with bcrypt-hashed passwords.
- Login: `POST /api/auth/login` checks the password, signs a JWT (`jsonwebtoken`), sets the `debtrex_session` httpOnly cookie. 8-hour expiry.
- The Edge middleware verifies the JWT using `jose` (Edge runtime can't run `jsonwebtoken`'s Node crypto). The API routes verify with `jsonwebtoken` because they run on Node.
- The JWT payload carries `id`, `email`, `role`, `full_name` so we don't need a DB hit just to know who you are. We *do* re-fetch the profile in `getCurrentUser()` to honor role changes and `is_active` flips mid-session.

External integrations

Service	What for	Where
Supabase (Postgres)	All data storage	<code>lib/supabase.ts</code>
Resend	Outbound email — task assigns, deadline alerts, invites	<code>lib/email.ts</code>
Google Drive	Per-user file storage (OAuth, refresh tokens stored on profile)	<code>lib/google-drive.ts</code> , <code>app/api/google/*</code>
Twilio Voice	Browser-based outbound + inbound calls, recording, AMD, voicemail	<code>lib/twilio.ts</code> , <code>lib/twilio-voice.ts</code> , <code>app/api/twilio/*</code>

The "permissions" layer

There are three things called "permissions" in this codebase. Don't confuse them.

1. **Role helpers** in `lib/roles.ts` — hard-coded checks like `canViewAllTasks(role)`. Used for fallback / built-in roles where the answer is constant.
2. **The `role_permissions` table** — every (`role`, `permission_key`) pair with an `enabled` bool. CEO can toggle these in the `/permissions` UI. Checked via `hasPermission()`.
3. `/api/me/permissions` — endpoint that returns the calling user's flat permission map, cached client-side in `sessionStorage` for 5 minutes so the Sidebar doesn't refetch on every nav.

When you add a feature: hardcode the fallback in `lib/roles.ts` for built-in roles, **and** add the granular key in `lib/permissions.ts` and seed it in a migration. The granular toggle wins if it exists.

Folder responsibilities

Folder	Rules
<code>app/</code>	Pages and API routes only. Server components by default — only mark "use client" when you need state/effects/handlers.

Folder	Rules
components/	Reusable client components. None of these should query Supabase directly — they receive data as props or call API routes.
lib/	Server-only utilities. Most files import 'server-only' at the top so they fail loudly if imported from a client component. The exception is lib/roles.ts, which is pure functions safe to import from both sides.
database/	schema.sql is the initial install. Anything after is migration-NNN.sql, applied in order via the Supabase SQL editor.

What doesn't exist (don't look for it)

- **No Redux / Zustand / global store.** Each page is a server render; client state is local.
- **No tRPC or GraphQL.** Plain fetch to /api/... routes.
- **No ORM.** Direct Supabase client queries.
- **No tests.** (Yet.)
- **No RLS.** Authorization is in app code.

PART 8

Data Model

Source: docs/data-model.md

Data Model

All tables live in the public schema in Supabase. The canonical schema is `database/schema.sql`; tables added later live in `database/migration-NNN.sql` files.

Entity overview

```

profiles ■■■< tasks ■< task_updates
          ■          ■< task_notes
          ■■■< events
          ■■■< expenses ■approved_by■■■
          ■
          ■■■< income          ■
          ■■■< budgets          ■
          ■■■< pipeline_contacts ■< call_logs
          ■
          ■          ■< contact_notes
          ■          ■< pipeline_collaborators
          ■■■< manager_assignments (self-join: manager_id, agent_id)
          ■■■< notifications
          ■■■< audit_log
          ■■■< calculations (saved calculator results – migration-004)

role_permissions (role × permission_key → enabled bool)
custom_roles     (role registry – built-in flags + custom additions)
phone_number_routing (Twilio inbound routing config)
  
```

Tables

profiles — users

Column	Notes
id (uuid, PK)	
email (unique)	Login identity
password_hash	bcrypt, cost 10
full_name	
role	Stored as text — both built-in (ceo, owner, co-owner, manager, employee, accountant, viewer) and custom role keys live here. The CHECK constraint was dropped in migration-005 so custom roles work.
is_active	Toggle to soft-disable a user without deleting
google_drive_folder_id, google_access_token, google_refresh_token, google_token_expires_at	Per-user Drive OAuth
status, status_message, status_updated_at	Online/OTL/meeting/break/offline presence (migration-007)

Column	Notes
notification_preferences	jsonb — opt-out flags for email categories

Indexes: email, role, partial status WHERE is_active = true.

tasks — work items

Owned by created_by, worked on by assigned_to. Status: not_started → in_progress → submitted → completed, plus overdue and cancelled. Priority is low/medium/high/urgent. Indexed on assigned_to, created_by, status, deadline.

task_updates — work log

Hours, status changes, free-text. Used to compute performance metrics and "recent activity" feeds. Indexed on task_id.

task_notes

Short discussion thread on a task. No index — query goes through task_id only and the table is small.

events — calendar

Includes visibility scope (private/team/company), color, all-day flag. Indexed on start_time. Calendar pages should always query with a date window — the page reads ±6 months around today.

event_attendees

M:N junction between events and profiles. Composite PK (event_id, user_id).

budgets — monthly allocations

Per (month, category) — month is a date truncated to the first of the month. Unique on (month, category).

expenses

Submitter is paid_by, approver is approved_by. Status: submitted → approved → reimbursed (or rejected). Indexed on paid_by, status, expense_date.

income

Recorded against a category and date. No FK on category — free text. Indexed on received_date.

audit_log

Append-only trail. action, resource_type, resource_id, plus details jsonb and ip/user-agent.

notifications

In-app notification queue. is_read flag. Composite index on (user_id, is_read) for fast unread-count.

`role_permissions` — granular auth (migration-003)

One row per (role, permission_key) pair with an `enabled` bool. Unique on (role, permission_key). Seeded with sensible defaults; CEO toggles from /permissions UI. Read via `lib/permissions.ts`.

`custom_roles` — role registry (migration-005)

Lists every role usable in `profiles.role`. Built-in roles are seeded with `is_built_in = true`. Custom roles created via /roles admin UI default to `is_built_in = false` and start with all permissions OFF.

`pipeline_contacts` (migration-003)

Unified table for both pipelines. `pipeline_type` discriminates 'management' (B2B deals) from 'sales' (debt-relief leads). Different columns matter for each. Status is free text but distinct vocabularies per pipeline (see `app/pipeline/PipelineList.tsx`). Indexed on type, status, `assigned_to`, `next_followup_at`.

`call_logs` (migration-003 + 007)

Every call attempt. `direction` is `outbound` or `inbound`. `outcome` is one of a fixed list (extended in 007 to add the `inbound_*` cases). For inbound calls that didn't match a contact, `contact_id` is null and `from_number` carries the caller's number. Composite indexes on (`contact_id`, `called_at` desc) and (`user_id`, `called_at` desc) and (`direction`, `called_at` desc).

`contact_notes` (migration-003)

Sticky notes on a contact, separate from per-call notes. `is_pinned` flag.

`pipeline_collaborators` (migration-005)

Lets multiple people share a contact beyond the single `assigned_to`. Either `collaborator` (full edit) or `observer` (read-only). Unique on (`contact_id`, `user_id`).

`manager_assignments`

Self-referential M:N between profiles (`manager_id` ↔ `agent_id`). Used by the performance dashboard so managers only see their assigned agents.

`calculations` (migration-004)

Saved calculator results — debt scenarios. Owned by `user_id`.

`phone_number_routing` (migration-007)

For each Twilio number, where inbound calls go:

- `agent` → ring a specific person
- `available` → ring all online agents
- `voicemail` → straight to voicemail

Conventions

- **Primary keys are uuid** (`uuid_generate_v4()`). The `uuid-oss` extension must be enabled.
- **Timestamps** are `timestampz`, defaulted to `now()`. `updated_at` is auto-bumped via the `update_updated_at_column()` trigger on tables that need it.
- **Money** is `numeric(12,2)`. Don't use float.
- **Soft delete** is `is_active = false` on profiles. Other tables are hard-deleted (often via `ON DELETE CASCADE`).
- **Status / enum fields** are stored as `text` with CHECK constraints. To add a new value you alter the constraint in a migration — see migration-007 for `call_logs.outcome` as the pattern.

Adding a column or table

1. Create `database/migration-NNN.sql` (next number).
2. Write `alter table ... add column if not exists ...` or `create table if not exists ....`
3. Add any needed indexes — usually one per `eq()` filter column the app uses.
4. Open Supabase SQL editor → paste → run.
5. Update relevant TypeScript types — there are no generated types in this repo, so you're updating literal types in `lib/auth.ts`, route handlers, etc.

PART 9

Auth & Permissions

Source: docs/auth-and-permissions.md

Auth & Permissions

This app does **not** use Supabase Auth. Auth is hand-rolled: bcrypt passwords, JWT cookies, application-level permission checks. RLS is off; the server is the only thing standing between a user and the database.

The session cookie

Cookie name	debtrex_session
Set by	POST /api/auth/login (success)
Cleared by	POST /api/auth/logout and clearSessionCookieOnResponse()
Value	JWT signed with JWT_SECRET (env var)
Claims	id, email, role, full_name
Lifetime	8 hours (expiresIn: '8h')
Flags	httpOnly, sameSite=lax, secure in production

Two JWT libraries, on purpose:

- jose in middleware.ts (Edge runtime — no Node crypto allowed).
- jsonwebtoken in lib/auth.ts (Node — used by API routes and Server Components).

Both verify against the same JWT_SECRET, so they're interchangeable.

Login flow

```
POST /api/auth/login { email, password }
  ↓ look up profile by email
  ↓ bcrypt.compare
  ↓ jwt.sign({ id, email, role, full_name }, JWT_SECRET, 8h)
  ↓ Set-Cookie: debtrex_session=<jwt>; HttpOnly; SameSite=Lax
  ← 200 { user }
```

First-CEO setup

POST /api/auth/setup { email, password, full_name } works **only when the profiles table is empty**. It creates the first CEO. After that the endpoint returns "setup already completed" and you create more users via /team (Team UI → POST /api/users/invite).

Middleware (route protection)

middleware.ts runs on every request. It allows through:

- Static assets (/_next/*, /favicon.ico)

- The login form and login API (`/login`, `/api/auth/login`, `/api/auth/setup`)
- The OAuth callback (`/api/google/callback`)
- Twilio webhooks (a whitelist of `/api/twilio/twiml`, `/status`, `/recording`, `/amd`, `/voice-twiml`, `/inbound`, `/inbound-fallback`, `/voicemail`) — these are POSTed by Twilio's servers, not the browser

Everything else needs a valid `debtrex_session` cookie. If absent or invalid:

- Page request → 302 to `/login`
- API request → 401 JSON

Important: middleware only verifies the JWT signature. It does NOT check `is_active` or current role — those checks live in `getCurrentUser()` at the page/route level.

`getCurrentUser()` — what every server entry does

```
const user = await getCurrentUser();
if (!user) redirect('/login');           // page
// or
if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 }); // route
```

What it does:

1. Read the cookie.
2. Verify the JWT.
3. Fetch the profiles row by id, selecting `id`, `email`, `full_name`, `role`, `google_drive_folder_id`, `is_active`.
4. Return null if `is_active = false`.

It's wrapped in React's `cache()` so multiple calls inside one request share the result. The DB hit happens once per request.

Roles

Built-in roles (defined in `lib/roles.ts`):

Role	Code constant
CEO	<code>ROLES.CEO</code>
Owner	<code>ROLES.OWNER</code>
Co-Owner	<code>ROLES.CO_OWNER</code>
Manager	<code>ROLES.MANAGER</code>
Employee	<code>ROLES.EMPLOYEE</code>
Accountant	<code>ROLES.ACCOUNTANT</code>
Viewer	<code>ROLES.VIEWER</code>

Hard-coded helpers in `lib/roles.ts` answer questions like:

- `canManageUsers(role)`
- `canEditBudget(role)`
- `canApproveExpenses(role)`
- `canViewAllTasks(role)`
- `canCreateEvents(role)`
- `canDeleteUsers(role)` — CEO only
- `isLeadership(role)` — CEO / Owner / Co-Owner

These are pure functions — fine to import from both server and client.

Custom roles (created via `/roles`) live in `custom_roles`. Their `role_key` goes into `profiles.role` and starts with **all permissions OFF**. The built-in role helpers won't recognize them — only `hasPermission()` (the granular system) will.

The granular permission system

There are about 50 permission keys, organized like `section.tasks`, `task.create`, `expense.approve`, `pipeline.view_all`, etc. The full list lives in `lib/permissions.ts` and the seed list in `database/migration-003.sql`.

Each (role, key) pair has an `enabled` bool in `role_permissions`. CEO can toggle any of them on the `/permissions` admin page.

Checking permissions on the server

```
import { hasPermission } from '@lib/permissions';

if (!(await hasPermission(user.role, 'task.delete'))) {
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
}
```

Checking many permissions at once

```
import { getPermissions } from '@lib/permissions';
const perms = await getPermissions(user.role); // { 'task.delete': true, ... }
```

Caching

`hasPermission()` lazy-loads the full `role_permissions` table into an in-process `Map` and caches it for **5 minutes**. Cache invalidates automatically by TTL. When you change permissions via the admin API, call `invalidatePermissionCache()` to flush immediately.

The cache is per Node process. On Vercel, every cold-started function pays one DB query to fill the cache; warm requests don't.

Client-side permission checks

Don't trust them — they're for UX only. The Sidebar pre-fetches `/api/me/permissions` once per session, caches the result in `sessionStorage` for 5 minutes, and uses it to decide which nav items to show. The actual security comes from the server-side `hasPermission()` checks in the page/API handler.

Permission fallback pattern

Some pages combine both systems:

```
const allowed = await hasPermission(user.role, 'section.budget')
  || canEditBudget(user.role); // built-in fallback
```

This means: the granular permission wins if it's set; otherwise the hard-coded built-in helper is used as a fallback. Use this when a new permission key was added but might not be seeded for older roles.

Adding a new permission

1. **Add the key** to `PERMISSION_KEYS` in `lib/permissions.ts`.
2. **Add a label** in `PERMISSION_LABELS` so the admin UI shows a friendly name.
3. **Put it in a group** in `PERMISSION_GROUPS` so it appears under the right section.
4. **Seed it** in a new `database/migration-NNN.sql` — copy the `seed_permission` pattern from `migration-003` (or `seed_perm_if_missing` from `migration-006`).
5. **Use it** in the relevant route handler: `await hasPermission(user.role, 'your.new.key')`.
6. After deploying, run the migration in Supabase SQL editor.

Adding a new role

Use the `/roles` admin UI. No code change needed — custom roles work the same as built-ins for the granular permission system. They just won't satisfy the built-in `lib/roles.ts` helpers like `isLeadership()`, so anything that uses those helpers will treat custom roles as "not leadership."

PART 10

API Reference

Source: docs/api-reference.md

API Reference

Every route lives under `app/api/<path>/route.ts`. The file's exported function names (`GET`, `POST`, `PATCH`, `DELETE`) define which HTTP methods it accepts. All routes (except those whitelisted in `middleware.ts`) require a valid `debtrex_session` cookie.

Status codes used: `200` (success), `400` (bad input), `401` (no session), `403` (forbidden), `404` (not found), `500` (server error).

Auth

Method	Path	Purpose	Auth
POST	<code>/api/auth/login</code>	Validate email/password, set session cookie	Public
POST	<code>/api/auth/logout</code>	Clear session cookie	Public
POST	<code>/api/auth/setup</code>	Create first CEO. Only works when no users exist.	Public

Current user (me)

Method	Path	Purpose	Auth
GET	<code>/api/me/permissions</code>	Return calling user's flat permission map	Any signed-in
GET	<code>/api/me/status</code>	Get my status + team status list	Any signed-in
PATCH	<code>/api/me/status</code>	Update my online/OTL/meeting/break/offline status	Any signed-in
POST	<code>/api/me/status</code>	Same as PATCH (alt for navigator.sendBeacon)	Any signed-in
PATCH	<code>/api/users/me</code>	Update my profile (name, phone, etc.)	Any signed-in
PATCH	<code>/api/users/me/password</code>	Change my password (requires current password)	Any signed-in
POST	<code>/api/users/me/google/disconnect</code>	Revoke my stored Google Drive tokens	Any signed-in

Tasks

Method	Path	Purpose	Permission
POST	/api/tasks	Create a task	task.create
PATCH	/api/tasks/[id]	Update title/status/deadline/etc.	task.edit_any or task ownership
POST	/api/tasks/[id]/updates	Add a work-log update (hours, status)	Assignee
POST	/api/tasks/[id]/notes	Add a discussion note	Assignee or creator

Task deletion is not currently exposed via API — tasks are kept for history. Use status `cancelled` to retire one.

Events / Calendar

Method	Path	Purpose	Permission
POST	/api/events	Create a calendar event (+ attendees)	event.create

Edit / delete of events are not currently exposed via API; create-only workflow.

Pipeline (Management + Sales)

Method	Path	Purpose	Permission
POST	/api/pipeline	Create a contact (pass pipeline_type)	pipeline.create
PATCH	/api/pipeline/[id]	Update contact fields	pipeline.edit_any or owner/collaborator
DELETE	/api/pipeline/[id]	Delete contact	pipeline.delete
POST	/api/pipeline/[id]/calls	Log a manual call attempt	call.log
POST	/api/pipeline/[id]/notes	Add a sticky note	call.log (or owner)
PATCH	/api/pipeline/notes/[id]	Edit a note	Note author
DELETE	/api/pipeline/notes/[id]	Delete a note	Note author or leadership
GET	/api/pipeline/[id]/collaborators	List collaborators on a contact	Anyone with contact access

Method	Path	Purpose	Permission
POST	/api/pipeline/[id]/collaborators	Add a collaborator	Contact owner or pipeline.assign_to_anyone
DELETE	/api/pipeline/collaborators/[collabId]	Remove a collaborator	Contact owner or pipeline.assign_to_anyone

Budget

Method	Path	Purpose	Permission
POST	/api/budgets	Create a monthly allocation (month, category, amount)	budget.edit_allocations
DELETE	/api/budgets/[id]	Remove an allocation	budget.edit_allocations
POST	/api/budgets/copy-previous	Copy last month's allocations into this month	budget.edit_allocations
POST	/api/expenses	Submit an expense	expense.submit
PATCH	/api/expenses/[id]	Approve/reject/edit	expense.approve (for approvals)
POST	/api/income	Record income	income.record

Files (Google Drive)

Method	Path	Purpose	Permission
GET	/api/google/auth	Start OAuth — redirects to Google consent screen	Any signed-in
GET	/api/google/callback	OAuth landing — exchanges code, stores tokens on profile	Public (Google → our server)
POST	/api/files/upload	Upload a file to caller's Drive folder	Any signed-in with Drive connected
DELETE	/api/files/[id]	Delete a file from Drive	File owner

Users / Team

Method	Path	Purpose	Permission
POST	/api/users/invite	Create a new user, send invite email with temp password	team.invite
PATCH	/api/users/[id]	Update role / active flag / fields	team.change_roles / team.deactivate
PATCH	/api/users/[id]/twilio-number	Assign a Twilio number to a user	Leadership

Roles (custom)

Method	Path	Purpose	Permission
POST	/api/roles	Create a custom role	roles.manage
PATCH	/api/roles/[key]	Rename / re-color a custom role	roles.manage
DELETE	/api/roles/[key]	Delete a custom role (built-ins protected)	roles.manage

Permissions admin

Method	Path	Purpose	Permission
POST	/api/permissions	Bulk-update the role_permissions matrix	permissions.manage

After any change, the server calls `invalidatePermissionCache()` so the next request reads fresh data.

Performance / Manager assignments

Method	Path	Purpose	Permission
POST	/api/manager-assignments	Assign an agent to a manager	performance.assign_managers
DELETE	/api/manager-assignments/[id]	Remove an assignment	performance.assign_managers

Calculators

Method	Path	Purpose	Permission
POST	/api/calculations	Save a calculator result	Any signed-in
PATCH	/api/calculations/[id]	Rename / edit a saved result	Owner
DELETE	/api/calculations/[id]	Delete a saved result	Owner

Twilio (browser-call related)

Method	Path	Purpose	Auth
GET	/api/twilio/voice-token	Mint a short-lived JWT for the browser Voice SDK	call.make
POST	/api/twilio/call	Initiate a server-side outbound call (non-browser)	call.make
GET	/api/twilio/diagnostics	Check Twilio config (returns booleans, no secrets)	Leadership
POST	/api/twilio/log-browser-call	Persist call_log row after a browser Voice call ends	Any signed-in
POST	/api/twilio/routing	Configure inbound routing for a number	Leadership
GET	/api/twilio/recordings/[id]	Stream a recording (proxied with auth)	Call participants or leadership

Twilio webhooks — these are PUBLIC (called by Twilio's servers)

These are whitelisted in `middleware.ts`. **In production, validate the `X-Twilio-Signature` header** before trusting the body (currently not enforced — see TODO in `middleware.ts`).

Method	Path	Purpose
POST, GET	/api/twilio/twiml/dial	Twiml for outbound bridge
POST	/api/twilio/status	Call lifecycle status callbacks
POST	/api/twilio/recording	Recording-ready callback
POST	/api/twilio/amd	Answering-machine detection result
POST, GET	/api/twilio/voice-twiml	Inbound Twiml for browser identity

Method	Path	Purpose
POST, GET	/api/twilio/inbound	Primary inbound webhook — selects routing target
POST, GET	/api/twilio/inbound-fallback	If inbound fails
POST	/api/twilio/voicemail	Voicemail recording callback

Conventions for new routes

```
// app/api/<feature>/route.ts
import { NextResponse } from 'next/server';
import { getCurrentUser } from '@lib/auth';
import { hasPermission } from '@lib/permissions';
import { supabaseAdmin } from '@lib/supabase';
import { z } from 'zod';

const BodySchema = z.object({
  title: z.string().min(1),
  // ...
});

export async function POST(request: Request) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  if (!(await hasPermission(user.role, 'task.create'))) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const body = BodySchema.safeParse(await request.json());
  if (!body.success) return NextResponse.json({ error: body.error.issues }, { status: 400 });

  const { data, error } = await supabaseAdmin
    .from('tasks')
    .insert({ ...body.data, created_by: user.id })
    .select()
    .single();

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json(data);
}
```

Three things to remember:

1. **Auth check first**, before any DB work.
2. **Validate input** with zod. Don't trust the body shape.
3. **Use** `supabaseAdmin` server-side. The anon client is for client-side reads that don't exist much in this app.

PART 11

UI & Styling

Source: docs/ui-and-styling.md

UI & Styling Guide

How the website *looks* — colors, fonts, reusable CSS classes, component patterns, mobile rules.

If you want to change visual design (a button color, a font, the brand red), this is the doc.

1. The brand

Token	Hex	Tailwind class
Brand Red	#E02020	bg-brand-red, text-brand-red, border-brand-red
Brand Red Dark (hover)	#B81414	bg-brand-red-dark
Brand Red Light	#FF3B3B	bg-brand-red-light
Brand Red Pale (backgrounds)	#FFF0F0	bg-brand-red-pale
Ink (primary text)	#111111	text-brand-ink, bg-brand-ink
Ink-2 (secondary text)	#333333	text-brand-ink-2

The palette lives in `tailwind.config.js` under `theme.extend.colors.brand`. Add a new shade there if needed.

Fonts

- **Default / body** — Barlow. Class: `font-sans` (default — you don't have to write it).
- **Display / condensed** — Barlow Condensed Black. Class: `font-condensed`. Use for big headings, KPI numbers, hero text.

Both are loaded via Google Fonts in `app/globals.css` at the top.

2. Reusable classes (defined in `globals.css`)

These are **NOT** Tailwind built-ins. They're our own component classes built with `@apply`. Use them everywhere for visual consistency.

Buttons

```
<button class="btn-primary">Save</button>      <!-- red, white text -->
<button class="btn-outline">Cancel</button>    <!-- ink border, transparent fill -->
<button class="btn-ghost">View all</button>    <!-- text-only, no border -->
```


All buttons have a **minimum 42px tap target** so they work on mobile.

Card

```
<div class="card p-6">
  <!-- content -->
</div>
```

White background, rounded, light shadow. Apply your own padding (p-6, etc.).

Form inputs

```
<label class="label">Email</label>
<input class="input" type="email" />
```

- `.label` — small, bold, uppercase, tracked-out gray text.
- `.input` — full-width, bordered, 16px font on mobile (prevents iOS zoom on focus).

Badges (small pills)

```
<span class="badge badge-red">Overdue</span>
<span class="badge badge-green">Completed</span>
<span class="badge badge-blue">In progress</span>
<span class="badge badge-yellow">Submitted</span>
<span class="badge badge-gray">Not started</span>
```

Headings

```
<h1 class="heading-condensed text-3xl">PIPELINE</h1>
```

Short for `font-condensed font-black uppercase tracking-tight`.

Animations

```
<div class="fade-in">Appears with a 300ms fade-up.</div>
```

Scrollbars

- Custom red-on-hover scrollbar is global.
- `.no-scrollbar` hides scrollbars while keeping scroll behavior — useful for horizontal filter rows.

3. The page shell (every page uses this)

Every authenticated page has the same outer shape:

```
<div className="flex min-h-screen bg-gray-50">
  <Sidebar user={user} />
  <main className="flex-1 min-w-0">
    <TopBar user={user} title="My Page Title" />
    <div className="p-4 sm:p-6 max-w-7xl mx-auto">
      {/* page content goes here */}
    </div>
  </main>
</div>
```

```
</main>
</div>
```

- `min-h-screen` ensures the sidebar fills the viewport.
- `flex-1 min-w-0` on the main column lets it grow without overflowing.
- `max-w-7xl mx-auto` keeps content from getting too wide on big monitors.
- `p-4 sm:p-6` is the standard padding.

Change the max width? Wider lists usually use `max-w-7xl`. Forms / detail pages use `max-w-3xl`. Login uses `max-w-md`.

4. Sidebar — navigation

Lives in `components/Sidebar.tsx`. Black background, white text, red accents.

Navigation is **grouped** (Home / Pipeline / Tools / Team / Financial / Admin). Each group has its own collapse state, persisted to `localStorage`.

Each `NavItem` has:

```
{ href, label, icon, show }
```

The `show` boolean comes from the `can('permission.key', fallbackBool)` helper:

- If permissions have loaded, it uses the granular map.
- If not, it uses the fallback (so the UI doesn't flicker).

To add a new link, see [recipes.md](#) → **R27**.

5. TopBar — header

Lives in `components/TopBar.tsx`. White, sticky-ish.

- Page title (passed as prop).
- Status dropdown (Online / OTL / Meeting / Break / Offline).
- Bell icon (notifications) — currently linked, dropdown rendering may be in a subcomponent.
- Avatar with initials.

Status values are defined in the `STATUSES` array at the top of `TopBar.tsx`. To add one, also extend the `CHECK` constraint on `profiles.status` via a migration.

6. Icons

We use [lucide-react](#) exclusively. Import only what you need:

```
import { CheckSquare, Plus, AlertCircle } from 'lucide-react';
<CheckSquare size={16} />
```

`next.config.js` has `optimizePackageImports: ['lucide-react']` so unused icons are tree-shaken out.

Common sizes:

- 12 — inside small buttons (`btn-ghost`).
- 14 — `btn-primary`/`btn-outline`.
- 16 — sidebar nav, generic.
- 20 — `TopBar`.
- 24 — section headers.
- 32 — empty states.

7. Color conventions

Purpose	Class
Primary action button	<code>bg-brand-red text-white</code>
Secondary action button	<code>border-brand-ink text-brand-ink</code>
Error / overdue	<code>text-brand-red / bg-brand-red-pale</code>
Success	<code>text-green-700 / bg-green-50</code>
Warning	<code>text-yellow-700 / bg-yellow-50</code>
Info	<code>text-blue-700 / bg-blue-50</code>
Subtle background	<code>bg-gray-50</code>
Card border	<code>border-gray-200</code>
Body text	<code>text-gray-900</code> (default ink)
Muted text	<code>text-gray-500 / text-gray-600</code>
Placeholder/disabled	<code>text-gray-400</code>

8. Spacing scale (Tailwind defaults)

Class	px
p-1	4
p-2	8
p-3	12
p-4	16
p-5	20
p-6	24
p-8	32

Same for m-, gap-, space-x-/space-y-.

9. Responsive (mobile-first)

Tailwind classes without a prefix apply on all screen sizes. Prefixes mean "from this size up":

- sm: ≥ 640px
- md: ≥ 768px
- lg: ≥ 1024px
- xl: ≥ 1280px

Typical pattern in this codebase:

```
<div class="p-4 sm:p-6 max-w-7xl mx-auto" >      <!-- tighter padding on phones -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4" >  <!-- responsive columns -->
<div class="hidden lg:block" >      <!-- desktop only -->
<div class="lg:hidden" >      <!-- mobile only -->
```

The sidebar collapses to a hamburger menu under lg: (1024px).

10. Forms

Standard form pattern:

```
<form onSubmit={submit} className="space-y-4">
  <div>
    <label className="label">Title</label>
    <input className="input" value={title} onChange={e => setTitle(e.target.value)} />
  </div>
  <div>
    <label className="label">Description</label>
    <textarea className="input" rows={4} value={desc} onChange={e => setDesc(e.target.value)} />
  </div>
  <div className="flex justify-end gap-2">
```

```

    <button type="button" className="btn-outline" onClick={onCancel}>Cancel</button>
    <button type="submit" className="btn-primary" disabled={saving}>
      {saving ? 'Saving...' : 'Save'}
    </button>
  </div>
</form>

```

- `space-y-4` gives consistent vertical spacing between fields.
- Always show a disabled state on the submit button while saving — disabled + swapping label is the cheapest UX win.

11. Modals

There's no shared `<Modal>` component — each modal is inlined. The standard pattern:

```

{open && (
  <div className="fixed inset-0 z-50 bg-black/50 flex items-center justify-center p-4">
    <div className="bg-white rounded-lg max-w-md w-full p-6 fade-in">
      <h2 className="heading-condensed text-xl mb-4">New widget</h2>
      { /* form */ }
    </div>
  </div>
)}

```

- `fixed inset-0` covers the screen.
- `z-50` puts it above the sidebar.
- `bg-black/50` is the backdrop.
- `flex items-center justify-center` centers the panel.
- `max-w-md w-full` makes it responsive.

Add `onClick={() => setOpen(false)}` to the backdrop if you want click-outside-to-close.

12. Empty states

```

<div className="text-center py-10">
  <CheckSquare size={32} className="mx-auto text-gray-300 mb-3" />
  <h3 className="font-semibold text-gray-700">No tasks yet</h3>
  <p className="text-sm text-gray-500 mt-1">Create one to get started.</p>
  <Link href="/tasks/new" className="btn-primary mt-4">+ Create Task</Link>
</div>

```

The dashboard's `EmptyState` component is a reusable version of this — copy it if you're building lots of empty-state UI.

13. Loading states

For full pages, Next.js looks for `loading.tsx` in the same folder as `page.tsx`. Drop in a skeleton:

```
// app/tasks/loading.tsx
export default function Loading() {
  return (
    <div className="flex min-h-screen bg-gray-50">
      <div className="w-64 bg-brand-ink" />      {/* sidebar placeholder */}
      <main className="flex-1 p-6">
        <div className="h-8 w-48 bg-gray-200 rounded mb-6 animate-pulse" />
        <div className="space-y-3">
          {[1,2,3].map(i => <div key={i} className="h-20 bg-gray-200 rounded animate-pulse" />)}
        </div>
      </main>
    </div>
  );
}
```

For inline (button) loading, just swap the label: `{saving ? 'Saving...' : 'Save'}`.

14. Adding a new color or component class

1. **A new color:** edit `tailwind.config.js`:

```
colors: {
  brand: {
    red: '#E02020',
    gold: '#D4A017',    // new
  },
},
```

Now `text-brand-gold`, `bg-brand-gold`, etc. work everywhere.

2. **A new reusable class:** edit `app/globals.css`:

```
.btn-success {
  @apply inline-flex items-center gap-2 bg-green-600 text-white font-semibold px-4 py-2.5 rounded-md ho
  min-height: 42px;
}
```

Use it like any other class: `<button className="btn-success">...</button>`.

15. Don't reach for these (anti-patterns)

- **Don't use inline** `style={...}` unless you genuinely need a dynamic value (computed color, computed width). For static styles use Tailwind classes.
- **Don't add a CSS module** (`.module.css`) — we're 100% on utility classes + globals.
- **Don't install another UI kit** (MUI, shadcn, Chakra) — we hand-build to keep the bundle small. The card / button / input classes cover 95% of needs.

- **Don't use `<a>` for in-app links** — use `<Link from 'next/link'>` so client-side nav works and prefetching kicks in.

PART 12

Developer Workflow

Source: docs/dev-workflow.md

Developer Workflow

Running locally

```
npm install
cp .env.example .env.local      # fill in the values
npm run dev                     # → http://localhost:3000
```

Required env vars (full list in `.env.example`):

Var	Where it comes from
NEXT_PUBLIC_SUPABASE_URL	Supabase project settings → API
NEXT_PUBLIC_SUPABASE_ANON_KEY	same
SUPABASE_SERVICE_ROLE_KEY	same — keep secret
JWT_SECRET	<code>openssl rand -base64 32</code>
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI, GOOGLE_DRIVE_ROOT_FOLDER_ID	Google Cloud Console
RESEND_API_KEY, EMAIL_FROM	Resend dashboard
TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_API_KEY, TWILIO_API_SECRET, TWILIO_TWIML_APP_SID	Twilio console
NEXT_PUBLIC_APP_URL	<code>http://localhost:3000</code> for dev

You can skip Twilio / Resend / Google in dev — those features will degrade gracefully (Twilio routes 503, emails print to console if `RESEND_API_KEY` is unset).

First-run database setup

Paste `database/schema.sql` into Supabase SQL Editor → Run. Then run every `migration-NNN.sql` in order. After that:

```
curl -X POST http://localhost:3000/api/auth/setup \
  -H "Content-Type: application/json" \
  -d '{"email":"ceo@example.com","password":"ChangeMe123!","full_name":"Your Name"}'
```

This endpoint is the only way to create the first CEO. After that it's locked.

Common tasks

Add a page

1. Create `app/<route>/page.tsx`. It's a Server Component by default.
2. Boilerplate:

```
import { redirect } from 'next/navigation';
import { getCurrentUser } from '@lib/auth';
import { hasPermission } from '@lib/permissions';
import { supabaseAdmin } from '@lib/supabase';
import Sidebar from '@components/Sidebar';
import TopBar from '@components/TopBar';

export default async function MyPage() {
  const user = await getCurrentUser();
  if (!user) redirect('/login');

  if (!(await hasPermission(user.role, 'section.mything'))) redirect('/');

  // Parallel-fetch everything you need
  const [aRes, bRes] = await Promise.all([
    supabaseAdmin.from('a').select('...'),
    supabaseAdmin.from('b').select('...'),
  ]);

  return (
    <div className="flex min-h-screen bg-gray-50">
      <Sidebar user={user} />
      <main className="flex-1 min-w-0">
        <TopBar user={user} title="My Page" />
        <div className="p-4 sm:p-6 max-w-7xl mx-auto">
          {/* content */}
        </div>
      </main>
    </div>
  );
}
```

3. Add to the Sidebar's nav groups in `components/Sidebar.tsx` if it should appear there.

Add an API route

See [api-reference.md § Conventions](#). The three rules: auth first, validate input, use `supabaseAdmin`.

Add a database column

```
# 1. Find the next migration number
ls database/
# 2. Create database/migration-008.sql
# 3. alter table xxx add column if not exists ...;
# 4. Open Supabase SQL editor → run
# 5. Update TS types where the column is read
```

Don't edit `schema.sql` — that's the install-from-scratch snapshot.

Add a permission

Walk through is in [auth-and-permissions.md § Adding a new permission](#).

Switch between client and server

You need...	Component type
Just rendering data from props	Server (default — no directive)
useState, useEffect, event handlers, hooks	Client ("use client" at top of file)
Fetch data	Prefer server. If a client component needs data, call an API route.
Pass data from server → client	Just pass it as props in JSX.

Client components **cannot** import anything from a file that starts with `import 'server-only'` (`lib/auth.ts`, `lib/permissions.ts`, `lib/email.ts`, `lib/google-drive.ts`, `lib/twilio*.ts`, `lib/performance.ts`). If you need role helpers in the browser, import from `lib/roles.ts` (which has no `server-only` import).

Gotchas you'll hit

- Cannot find module **after changing files** — restart `npm run dev`. The compiler caches aggressively.
- **Login works then immediately redirects back to /login** — `JWT_SECRET` changed between sign and verify. Sign in again after changing it. In dev, also clear the cookie.
- **"Forbidden" on a page that should be allowed** — check `role_permissions` for that role + key. Then check `lib/roles.ts` for the built-in helper that the page uses. Then check the route's permission check.
- Cannot read property 'id' of null **server-side** — you forgot `getCurrentUser()` and the redirect.
- **Twilio webhook 403** — middleware needs to whitelist the path. Check `TWILIO_WEBHOOK_PREFIXES` in `middleware.ts`.
- **Emails not arriving** — when `RESEND_API_KEY` is empty or invalid, emails are silently logged to the console. Check the dev server output.
- **Google Drive not connected for a user** — the user has to visit `/files` and click "Connect Google Drive" themselves. Their token is stored on their profile row.
- **server-only imported from client component** — error like `Module not found: Can't resolve 'next/headers'`. Move the import to a server file and pass data as props.

Build & deploy

```
npm run build    # compiles + page-by-page bundle stats in the output
npm start       # serve the built app
```

`next.config.js` currently sets `typescript.ignoreBuildErrors: true` and `eslint.ignoreDuringBuilds: true` — meaning TS errors won't block a build. That's pragmatic but means TS errors silently land in prod. When time allows, flip these off and fix the lingering errors.

Deploy: push to main; Vercel auto-deploys. Set all `.env.local` values as Vercel Environment Variables. Update `GOOGLE_REDIRECT_URI` and `NEXT_PUBLIC_APP_URL` to the production domain, and add the prod redirect URI to your Google OAuth Client.

Reading the codebase

- Start at `middleware.ts` to see what happens to every request.
- Then `lib/auth.ts` and `lib/permissions.ts` — the two files you'll come back to most.
- Then `app/page.tsx` (the dashboard) — it's the simplest example of the standard page pattern.
- Then `app/api/tasks/route.ts` and `app/api/tasks/[id]/route.ts` — the canonical API route shape.
- The pipeline (`app/pipeline/*` + `app/api/pipeline/*`) is the most complex module — read it last when you're comfortable.

PART 13

Performance Notes

Source: docs/performance.md

Performance Notes

This doc tracks what's known about app performance — what was slow, what was fixed, and what's worth doing next when you have time.

Mental model

The app's hot path on a page view is:

1. **Middleware** — JWT verify (cheap, in-memory).
2. `getCurrentUser()` — 1 DB query (profile lookup), `react.cache()`d per request.
3. **Page-level queries** — 1 to N Supabase calls. **These are usually the slowest step.**
4. **Render + stream HTML** — fast.
5. **Client hydrate** — Sidebar/TopBar/InboundCallListener mount, each doing their own `fetch()` calls.

Whenever the app feels slow, the question is almost always: *how many round-trips, and are they in parallel?*

What was fixed (May 2026)

Fix	Files	Why it mattered
Parallelized 3 serial DB queries on Dashboard	<code>app/page.tsx</code>	Cut page render time roughly in 3
Parallelized + date-windowed Calendar fetch	<code>app/calendar/page.tsx</code>	Was fetching every event/deadline ever; now ± 6 months. Same parallelization win.
Parallelized Task Detail fetch	<code>app/tasks/[id]/page.tsx</code>	task, updates, and notes now load together
Parallelized Performance metrics fetch	<code>lib/performance.ts</code>	Users / tasks / updates were serial — now in <code>Promise.all</code>
Parallelized Budget fetch + bounded income	<code>app/budget/page.tsx</code>	Income had no <code>limit</code> ; bounded to 200
Bounded Tasks list to 200 rows	<code>app/tasks/page.tsx</code>	Was unbounded — would grow forever
Trimmed <code>select('*')</code> to needed columns	dashboard, tasks list, calendar	Smaller payloads, less serialization
Removed wasted GET <code>/api/me/status</code> on every page nav	<code>components/TopBar.tsx</code>	Was fetching the entire team list, then immediately overwriting with PATCH
Mark-online runs once per session, not per nav	<code>components/TopBar.tsx</code>	Used <code>sessionStorage</code> flag

Fix	Files	Why it mattered
Re-enabled Next.js prefetch on sidebar links	components/Sidebar.tsx	Nav now warms up the next page on hover
Removed dead deps: recharts, react-email, @react-email/components, date-fns	package.json, next.config.js	Faster install, smaller node_modules

Run `npm install` after pulling these changes to drop the dead deps from your local `node_modules`.

What's still a thing

These are documented for the next perf pass — not urgent unless data grows.

`getCurrentUser()` does a DB query per request

`react.cache()` dedupes within a request, but every page load still incurs one `profiles` row lookup.

Options if this becomes a bottleneck:

- Trust the JWT for shell rendering (Sidebar/TopBar) and only DB-verify for sensitive actions.
- Add a short LRU cache (e.g. 30 sec per `user_id`) — profile changes are rare, and `is_active` toggles can wait that long.

The permission cache is per-process

`lib/permissions.ts` caches the `role_permissions` table in process memory for 5 minutes. On Vercel, every cold-started serverless function pays one DB fill. Per request, it's free after the first. Acceptable today.

N+1 risk on pipeline list

The pipeline list joins both `assigned_to_profile` and `created_by_profile`. As you grow past a few hundred contacts, that join will dominate. Consider denormalizing the displayed name onto the contact row, or paginating server-side.

FullCalendar + lucide-react bundle weight

FullCalendar is the heaviest client dep but already isolated to `/calendar` chunk (it's only imported in `CalendarView.tsx`). `next.config.js` opts it into `optimizePackageImports`. Lucide icons are tree-shaken via `optimizePackageImports`.

Inbound call listener fetches a token on every page

`InboundCallListener` is mounted from `Sidebar`, so every page nav fires `GET /api/twilio/voice-token`. Could be cached client-side (token has a TTL — re-issue only when expired). Not a huge cost but worth knowing.

Indexes that might matter at scale

These aren't bottlenecks at current data sizes, but consider when tables grow past tens of thousands of rows:

Table	Index	Why
tasks	composite (assigned_to, status)	Most "my open tasks" queries filter on both
tasks	partial (deadline) WHERE status NOT IN ('completed', 'cancelled')	Calendar/dashboard deadline lookups
events	composite (start_time, end_time)	Range queries from FullCalendar
task_updates	composite (user_id, created_at desc)	Performance "last activity"
expenses	composite (status, expense_date desc)	Budget dashboard listing
call_logs	already good — composite indexes on (contact_id, called_at) and (user_id, called_at)	

Always profile first. Adding an index has write-side cost.

How to measure if a page is slow

1. In Chrome DevTools → Network, hover the page document request. Note **TTFB** — that's the server-render time.
2. If TTFB is bad, the bottleneck is DB queries. Run `EXPLAIN ANALYZE` on the relevant query in Supabase SQL editor.
3. If TTFB is OK but the page still feels slow, it's bundle / hydration. Check the size of the page's JS chunk in `npm run build` output.

If you see a perf regression, the most common cause in this codebase is "someone added a `select('*')` join or a sequential `await` they didn't need to."

Dev-server only slowness

Next.js dev mode (`npm run dev`) is significantly slower than production. Reasons:

- Routes are compiled on first request, not ahead of time.
- HMR adds overhead.
- React Strict Mode (enabled in `next.config.js`) double-invokes effects in dev.

If a page feels slow in dev but fine after `npm run build && npm start`, it's a dev-mode artifact, not a real bug.

PART 14

Troubleshooting

Source: docs/troubleshooting.md

Troubleshooting

A flat list of every error / symptom we've seen, what causes it, and how to fix it. Use Ctrl-F.

Login & auth

"Invalid credentials" but the password is right

- Email is case-sensitive in some flows. `/api/auth/login` lowercases the input — but if you've imported users with mixed-case emails, the lookup will fail. Check `select email from profiles where lower(email) = '...';` in Supabase.
- The account may be inactive — `is_active = false`. You'll see "Account deactivated" instead, but if the user was just imported with bad data, it might be missing.

Login works but immediately redirects back to `/login`

- `JWT_SECRET` changed between when the cookie was signed and when it's verified. Restart `npm run dev` after changing `.env.local`. Then clear the cookie in DevTools and sign in again.
- The middleware uses `jose` (Edge) while the API uses `jsonwebtoken` (Node). Both verify against the same secret — if one library can't read it, you have an env-var issue. Look at the dev server console for `jwtVerify` errors.

"Setup already completed"

- `/api/auth/setup` only works when the `profiles` table is empty. To make a new user after that, sign in as CEO → `/team` → Invite.

Cookie disappears on every refresh

- In production, the cookie is `secure: true` and requires HTTPS. If you're testing on plain `http://`, the browser drops it. Use `https://` (Vercel sets this up automatically) or run in dev locally where `secure: false`.
-

Permissions

A user gets 403 on a page they should see

1. **Check the granular permission** — sign in as CEO → `/permissions` → toggle on the relevant key for their role.
2. **Check the code** — the page may be calling the wrong key, or using a hard-coded helper. Open the page file and look at the redirect line near the top.

3. **Check the cache** — `lib/permissions.ts` caches for 5 minutes. After toggling on `/permissions` the admin route calls `invalidatePermissionCache()`. If you changed something directly in SQL, the cache won't know — restart the server.

Custom role acts like it has no permissions

- Built-in helpers in `lib/roles.ts` are hard-coded for the 7 built-in role names. Custom roles always return `false` for them.
- The granular system (the `role_permissions` table) does work for custom roles, but newly created custom roles start with **all permissions OFF**. Go to `/permissions`, switch to that role column, toggle stuff on.

Sidebar shows the wrong items

- Sidebar uses `sessionStorage` to cache permissions for 5 minutes. After changing permissions, clear `sessionStorage` in DevTools or sign out and back in.
- Look at the `show` field in `components/Sidebar.tsx` for the misbehaving item — make sure the permission key matches what you seeded.

Database / Supabase

"uuid_generate_v4() does not exist"

Run `create extension if not exists "uuid-oss";` in Supabase SQL Editor.

"permission denied for table profiles"

You're querying with the anon client (`supabase`) instead of the admin client (`supabaseAdmin`). Server code should always use `supabaseAdmin`.

`select('*')` returns weird/extra columns

You added a column in a migration but didn't restart the dev server. Try `Ctrl+C` and `npm run dev` again.

"duplicate key value violates unique constraint"

Some table has a unique constraint (e.g. (`month`, `category`) on `budgets`). Use `.upsert()` instead of `.insert()` if you intend "insert OR update":

```
.upsert({...}, { onConflict: 'month,category' })
```

"null value in column 'foo' violates not-null constraint"

The DB requires this column. Either:

- Pass a value when inserting.
- Add a `default` in the schema.

- Make the column nullable in a migration.

`.single()` throws "JSON object requested, multiple (or no) rows returned"

- `.single()` requires exactly 1 row.
- Use `.maybeSingle()` for "0 or 1".
- For lists, drop both — Supabase returns an array by default.

Slow queries on Supabase

- Run `EXPLAIN ANALYZE` on the query in the SQL editor.
- Add an index — see [recipes.md → R5](#).
- Trim the columns in your `.select(...).select('*')` on a table with big text columns is expensive.

Next.js / React

"Cannot find module '@lib/auth'"

- Restart the dev server.
- Make sure `tsconfig.json` has the `@/*` path alias (should already be there).

"Module not found: Can't resolve 'next/headers'" (in a client component)

- You imported `lib/auth.ts` or another `server-only` module from a `"use client"` file.
- Fix: move that import to a server file (a `page.tsx`, a route handler) and pass the data down as props.

"Text content does not match server-rendered HTML" (hydration error)

- A client component is rendering something different on the server vs client. Common causes:
- Reading `localStorage` / `sessionStorage` during render (won't exist on server). Wrap reads in a `useEffect` or check `typeof window !== 'undefined'`.
- Using `new Date()` directly — server time $\neq$ client time. Convert to a string consistently.
- Conditional rendering based on `window` width — render the same thing on server and then update in `useEffect`.

Server Component throws "params should be awaited"

In Next.js 15+, `params` and `searchParams` are async:

```
export default async function Page({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
}
```

"Event handlers cannot be passed to Client Component props"

You forgot `"use client"` at the top of a component that uses `onClick` (or any handler).

Changes don't show up

- Restart the dev server. Next.js's HMR is good but not perfect.
- Delete `.next/` and restart for the nuclear option.
- Hard refresh in the browser (Ctrl+Shift+R / Cmd+Shift+R).

Build error "Type error: ..."

`next.config.js` has `typescript.ignoreBuildErrors: true` so the build technically passes despite TS errors. But the errors are real. Look at the dev server output, find the file, fix it.

Email (Resend)

Emails don't arrive

- Check `RESEND_API_KEY` is set in `.env.local`. If empty, emails are silently logged to the dev console (look there).
- Check the Resend dashboard → Logs. If you see "Domain not verified," your DNS records aren't right yet.
- Check spam / junk folder, especially for new domains.
- Make sure `EMAIL_FROM` matches a verified domain. You can't send from `noreply@yourdomain.com` until that domain is verified in Resend.

Got `403 Forbidden` from Resend

- The "from" address isn't on a verified domain.
 - Use `onboarding@resend.dev` as a temporary `EMAIL_FROM` for testing.
-

Google Drive

"Drive not connected" persists after Connect

- Open DevTools → Network. Did `/api/google/callback` succeed? If it errored, look at the response.
- Common cause: the redirect URI in Google Cloud Console doesn't exactly match `GOOGLE_REDIRECT_URI` in `.env.local`. Both must be identical, scheme and trailing slash included.

"Token expired" errors

- Google refresh tokens last ~6 months but can be revoked. The fix is to disconnect and reconnect: `/settings` → Disconnect Drive, then re-do the OAuth flow.

File upload fails with 401

- The user's stored token is dead. Same fix as above.
-

Twilio

Outbound calls fail with "Twilio not configured"

- Check all 5 Twilio env vars: `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_API_KEY`, `TWILIO_API_SECRET`, `TWILIO_TWIML_APP_SID`.
- `/api/twilio/diagnostics` returns booleans — visit it as leadership to see which one is missing.

Inbound calls don't ring the browser

- Is the user "online" in their status dropdown? Routing `available` only picks online agents.
- Is `phone_number_routing` configured for the number? Go to `/twilio-numbers` and set it.
- Check Twilio's debug logs in their console — look at the latest call and read the request/response trail.

Twilio webhook returns 403

- The path isn't whitelisted. Add it to `TWILIO_WEBHOOK_PREFIXES` in `middleware.ts`.

"Forbidden" on `/api/twilio/voice-token`

- The user doesn't have `call.make`. Toggle it on at `/permissions`.

Browser microphone doesn't work

- Calls require HTTPS. On `http://localhost:3000` it works (browsers allow localhost). On a real deployment without HTTPS it won't. Vercel gives you HTTPS by default.
-

Performance

A page loads slowly

1. Open DevTools → Network → the page document request.
2. Look at TTFB (Time To First Byte). If it's > ~500ms, the bottleneck is server-side data fetching.
3. The page is probably running `await` queries in series. Wrap them in `Promise.all`. See `docs/performance.md`.
4. Bound large lists with `.limit(200)` (or pagination).
5. Trim `.select('*')` to only the columns you actually render.

Bundle is too big / dev mode is slow

- Dev mode is significantly slower than prod. Run `npm run build && npm start` to compare.
 - Don't import heavy libraries from server-shared modules (`lib/`) — keep them in client components that are only loaded on the relevant page.
-

Build & deploy

Vercel build fails

- Check Vercel build logs for the exact error.
- Most common cause: an env var is missing in Vercel's Environment Variables.
- Note: TS errors do NOT block the build (`ignoreBuildErrors: true`). They show as warnings.

"Server selected an unsupported protocol"

- The redirect URI on Google OAuth doesn't include the production URL. Add the prod `https://...` redirect URI to your Google Cloud OAuth Client.

After deploy, user gets "Not Found" on the dashboard

- Cookies aren't getting set in production. Check that `NEXT_PUBLIC_APP_URL` matches the deployed domain.
 - Check that the cookie is actually being set — DevTools → Application → Cookies for your domain. The `debtrex_session` cookie should be there after login.
-

Quick "I broke it" rollback

```
git log --oneline -20      # find the commit before you broke it
git revert <bad-commit>    # creates a new commit that undoes the bad one
git push
```

Vercel will auto-deploy the revert within a minute or two.

For a DB migration that turned out wrong: write a new migration that reverses the change. Don't delete the old migration file from git.

Reset everything (nuclear option)

If you want to wipe your local environment and start fresh:

```
rm -rf node_modules .next
```

```
npm install  
npm run dev
```

To reset the database: in Supabase Dashboard → Database → Settings → "Reset database" (warning: destroys all data).